

part_a_classical

January 27, 2026

1 Classical Machine Learning for Video Actions

This script implements and compares three classical machine learning algorithms using extracted video features:

1. Support Vector Machine (Linear + RBF)
2. Random Forest Classifier
3. k-Nearest Neighbors (k-NN)

Features are extracted using existing project modules: - data_loader.py - feature_extraction.py

Author: Student_2024AB05275

```
[1]: pip install -r requirements.txt
```

```
Requirement already satisfied: numpy>=1.23 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 23)) (1.24.4)
```

```
Requirement already satisfied: scipy>=1.10 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 31)) (1.11.3)
```

```
Requirement already satisfied: torch>=2.0 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 42)) (2.3.0+cu121)
```

```
Requirement already satisfied: torchvision>=0.15 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 49)) (0.18.0+cu121)
```

```
Requirement already satisfied: openpyxl>=3.1.0 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 54)) (3.1.5)
```

```
Requirement already satisfied: opencv-python-headless in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 69)) (4.11.0.86)
```

```
Requirement already satisfied: scikit-learn>=1.3 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 77)) (1.8.0)
```

```
Requirement already satisfied: scikit-image in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 84)) (0.22.0)
```

```
Requirement already satisfied: matplotlib>=3.7 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 96)) (3.8.4)
```

```
Requirement already satisfied: seaborn>=0.13 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 103)) (0.13.2)
```

Requirement already satisfied: umap-learn in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 110)) (0.5.11)

Requirement already satisfied: tqdm>=4.66 in /opt/conda/lib/python3.11/site-packages (from -r requirements.txt (line 122)) (4.66.4)

Requirement already satisfied: filelock in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (3.13.1)

Requirement already satisfied: typing-extensions>=4.8.0 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (4.12.2)

Requirement already satisfied: sympy in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (1.12)

Requirement already satisfied: networkx in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (3.3)

Requirement already satisfied: jinja2 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (3.1.4)

Requirement already satisfied: fsspec in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (2024.2.0)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.1.105 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.105)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.1.105 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.105)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.1.105 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.105)

Requirement already satisfied: nvidia-cudnn-cu12==8.9.2.26 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (8.9.2.26)

Requirement already satisfied: nvidia-cublas-cu12==12.1.3.1 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.3.1)

Requirement already satisfied: nvidia-cufft-cu12==11.0.2.54 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (11.0.2.54)

Requirement already satisfied: nvidia-curand-cu12==10.3.2.106 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (10.3.2.106)

Requirement already satisfied: nvidia-cusolver-cu12==11.4.5.107 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (11.4.5.107)

Requirement already satisfied: nvidia-cuspars-cu12==12.1.0.106 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (12.1.0.106)

Requirement already satisfied: nvidia-nccl-cu12==2.20.5 in /opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt (line 42)) (2.20.5)

Requirement already satisfied: nvidia-nvtx-cu12==12.1.105 in

/opt/conda/lib/python3.11/site-packages (from torch>=2.0->-r requirements.txt
 (line 42)) (12.1.105)
 Requirement already satisfied: triton==2.3.0 in /opt/conda/lib/python3.11/site-
 packages (from torch>=2.0->-r requirements.txt (line 42)) (2.3.0)
 Requirement already satisfied: nvidia-nvjitlink-cu12 in
 /opt/conda/lib/python3.11/site-packages (from nvidia-cusolver-
 cu12==11.4.5.107->torch>=2.0->-r requirements.txt (line 42)) (12.1.105)
 Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in
 /opt/conda/lib/python3.11/site-packages (from torchvision>=0.15->-r
 requirements.txt (line 49)) (10.4.0)
 Requirement already satisfied: et-xmlfile in /opt/conda/lib/python3.11/site-
 packages (from openpyxl>=3.1.0->-r requirements.txt (line 54)) (2.0.0)
 Requirement already satisfied: joblib>=1.3.0 in /opt/conda/lib/python3.11/site-
 packages (from scikit-learn>=1.3->-r requirements.txt (line 77)) (1.4.2)
 Requirement already satisfied: threadpoolctl>=3.2.0 in
 /opt/conda/lib/python3.11/site-packages (from scikit-learn>=1.3->-r
 requirements.txt (line 77)) (3.5.0)
 Requirement already satisfied: imageio>=2.27 in /opt/conda/lib/python3.11/site-
 packages (from scikit-image->-r requirements.txt (line 84)) (2.34.2)
 Requirement already satisfied: tifffile>=2022.8.12 in
 /opt/conda/lib/python3.11/site-packages (from scikit-image->-r requirements.txt
 (line 84)) (2024.7.2)
 Requirement already satisfied: packaging>=21 in /opt/conda/lib/python3.11/site-
 packages (from scikit-image->-r requirements.txt (line 84)) (24.1)
 Requirement already satisfied: lazy_loader>=0.3 in
 /opt/conda/lib/python3.11/site-packages (from scikit-image->-r requirements.txt
 (line 84)) (0.4)
 Requirement already satisfied: contourpy>=1.0.1 in
 /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r
 requirements.txt (line 96)) (1.2.1)
 Requirement already satisfied: cycycler>=0.10 in /opt/conda/lib/python3.11/site-
 packages (from matplotlib>=3.7->-r requirements.txt (line 96)) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in
 /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r
 requirements.txt (line 96)) (4.53.0)
 Requirement already satisfied: kiwisolver>=1.3.1 in
 /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r
 requirements.txt (line 96)) (1.4.5)
 Requirement already satisfied: pyparsing>=2.3.1 in
 /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r
 requirements.txt (line 96)) (3.1.2)
 Requirement already satisfied: python-dateutil>=2.7 in
 /opt/conda/lib/python3.11/site-packages (from matplotlib>=3.7->-r
 requirements.txt (line 96)) (2.9.0)
 Requirement already satisfied: pandas>=1.2 in /opt/conda/lib/python3.11/site-
 packages (from seaborn>=0.13->-r requirements.txt (line 103)) (2.1.4)
 Requirement already satisfied: numba>=0.51.2 in /opt/conda/lib/python3.11/site-
 packages (from umap-learn->-r requirements.txt (line 110)) (0.63.1)

Requirement already satisfied: pynndescent>=0.5 in /opt/conda/lib/python3.11/site-packages (from umap-learn->-r requirements.txt (line 110)) (0.6.0)

Requirement already satisfied: llvmlite<0.47,>=0.46.0dev0 in /opt/conda/lib/python3.11/site-packages (from numba>=0.51.2->umap-learn->-r requirements.txt (line 110)) (0.46.0)

Requirement already satisfied: pytz>=2020.1 in /opt/conda/lib/python3.11/site-packages (from pandas>=1.2->seaborn>=0.13->-r requirements.txt (line 103)) (2024.1)

Requirement already satisfied: tzdata>=2022.1 in /opt/conda/lib/python3.11/site-packages (from pandas>=1.2->seaborn>=0.13->-r requirements.txt (line 103)) (2024.1)

Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.11/site-packages (from python-dateutil>=2.7->matplotlib>=3.7->-r requirements.txt (line 96)) (1.16.0)

Requirement already satisfied: MarkupSafe>=2.0 in /opt/conda/lib/python3.11/site-packages (from jinja2->torch>=2.0->-r requirements.txt (line 42)) (2.1.5)

Requirement already satisfied: mpmath>=0.19 in /opt/conda/lib/python3.11/site-packages (from sympy->torch>=2.0->-r requirements.txt (line 42)) (1.3.0)

Note: you may need to restart the kernel to use updated packages.

```
[1]: # =====
# Standard Library Imports
# =====
import os
import time
import psutil
import joblib
import tempfile
import warnings
from typing import Dict, Tuple

# =====
# Third-Party Imports
# =====
import numpy as np
import pandas as pd
from pathlib import Path
# Object-oriented, cross-platform file system path handling
import matplotlib.pyplot as plt

from evaluate import evaluate_classification

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV
```

```

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
)

from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from IPython.display import display, Markdown

# =====
# Project Imports
# =====
from feature_extraction import (
    visualize_classical_feature_importance,
    visualize_classical_feature_space,
    get_classical_feature_names
)

from data_loader import load_split_data_and_extract_features
from evaluate import create_full_comparison_table, generate_dynamic_observations

```

```

[2]: # =====
# Global Configurations
# =====
warnings.filterwarnings("ignore")
RANDOM_STATE = 42
CV_FOLDS = 5

# =====
# Utility Functions
# =====
def evaluate_model(
    model,
    X_test: np.ndarray,
    y_test: np.ndarray,
) -> Dict[str, float]:
    """
    Evaluate a trained model using multiple metrics.

    Returns:
        Dictionary containing accuracy, precision, recall, and F1-score.
    """

```

```

y_pred = model.predict(X_test)

return {
    "accuracy": accuracy_score(y_test, y_pred),
    "precision": precision_score(y_test, y_pred, average="macro"),
    "recall": recall_score(y_test, y_pred, average="macro"),
    "f1_score": f1_score(y_test, y_pred, average="macro"),
}

def plot_confusion_matrix(model, X_test, y_test, title: str) -> None:
    """
    Plot confusion matrix for a trained classifier.
    """
    cm = confusion_matrix(y_test, model.predict(X_test))
    disp = ConfusionMatrixDisplay(confusion_matrix=cm)
    disp.plot()
    plt.title(title)
    plt.show()

```

```

[3]: # =====
# Data Loading & Feature Extraction
# =====
print("\n Loading dataset splits and extract features...")

print("\n-----")
print("* Processing training dataset.....")
print("-----\n")
X_train, y_train, CLASS_TO_LABEL, LABEL_TO_CLASS = \
    ↪load_split_data_and_extract_features("train")

print("\n-----")
print("* Processing validation dataset.....")
print("-----\n")
X_val, y_val, _, _ = load_split_data_and_extract_features(split_name="val")

print("\n-----")
print("* Processing test dataset.....")
print("-----\n")
X_test, y_test, _, _ = load_split_data_and_extract_features(split_name="test")

print(f"\n\n* Train shape: {X_train.shape}")
print(f"* Test shape : {X_test.shape}")

```

Loading dataset splits and extract features...

* Processing training dataset...

Detected classes (5): ['class_1_Diving', 'class_2_JumpRope',
'class_3_SkyDiving', 'class_4_Surfing', 'class_5_Swing']

[INFO] * processing Video file : class_1_Diving/v_Diving_g12_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g09_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g10_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g19_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g14_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g15_c06.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g17_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g02_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g16_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g06_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g18_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g09_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g22_c06.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g06_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g03_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g02_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g12_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g16_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g15_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g19_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g04_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g15_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g13_c07.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g21_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g02_c06.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g01_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g12_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g24_c06.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g21_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g12_c07.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g13_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g21_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g23_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g17_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g06_c06.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g16_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g07_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g03_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g16_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g09_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g08_c05.avi

[illegible]

* Processing validation dataset...

Detected classes (5): ['class_1_Diving', 'class_2_JumpRope',
'class_3_SkyDiving', 'class_4_Surfing', 'class_5_Swing']

[INFO] * processing Video file : class_1_Diving/v_Diving_g07_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g22_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g14_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g21_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g06_c02.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g03_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g01_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g11_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g01_c06.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g15_c07.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g21_c07.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g11_c06.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g14_c03.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g18_c01.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g11_c04.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g23_c04.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g05_c04.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g15_c04.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g11_c03.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g16_c03.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g06_c02.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g07_c05.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g18_c03.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g21_c01.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g01_c07.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g25_c03.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g23_c04.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g01_c04.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g13_c01.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g07_c03.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g16_c04.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g13_c02.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g20_c04.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g18_c01.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g03_c03.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g17_c04.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g05_c07.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g21_c01.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g25_c02.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g19_c03.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g02_c01.avi

```

[INFO] * processing Video file : class_5_Swing/v_Swing_g08_c03.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g10_c04.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g17_c03.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g05_c05.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g23_c02.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g25_c04.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g15_c06.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g09_c06.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g02_c06.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g09_c07.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g01_c02.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g19_c04.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g09_c02.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g18_c07.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g22_c02.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g10_c04.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g05_c01.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g18_c04.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g08_c01.avi

```

```

-----
* Processing test dataset...
-----

```

```

Detected classes (5): ['class_1_Diving', 'class_2_JumpRope',
'class_3_SkyDiving', 'class_4_Surfing', 'class_5_Swing']

```

```

[INFO] * processing Video file : class_1_Diving/v_Diving_g04_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g14_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g13_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g24_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g09_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g17_c03.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g06_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g08_c06.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g19_c01.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g22_c04.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g03_c05.avi
[INFO] * processing Video file : class_1_Diving/v_Diving_g13_c05.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g25_c02.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g05_c05.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g14_c02.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g12_c04.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g11_c01.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g06_c03.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g09_c02.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g13_c02.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g09_c01.avi

```

```

[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g01_c01.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g03_c05.avi
[INFO] * processing Video file : class_3_SkyDiving/v_SkyDiving_g25_c01.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g24_c03.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g13_c05.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g14_c02.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g17_c05.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g17_c07.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g19_c03.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g24_c04.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g04_c03.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g08_c03.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g09_c01.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g10_c01.avi
[INFO] * processing Video file : class_4_Surfing/v_Surfing_g11_c04.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g23_c04.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g24_c05.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g06_c04.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g16_c01.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g12_c02.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g14_c02.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g07_c03.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g24_c01.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g04_c07.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g13_c02.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g07_c04.avi
[INFO] * processing Video file : class_5_Swing/v_Swing_g25_c01.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g21_c04.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g19_c01.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g12_c03.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g08_c03.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g04_c07.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g12_c05.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g07_c03.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g21_c05.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g05_c05.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g22_c06.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g16_c03.avi
[INFO] * processing Video file : class_2_JumpRope/v_JumpRope_g20_c02.avi

```

```

* Train shape: (280, 399)
* Test shape : (60, 399)

```

```

[4]: # -----
#  MEMORY UTILITY (PROCESS-LEVEL RAM)
#  -----

```

```

PROCESS = psutil.Process(os.getpid())

def get_ram_mb():
    """Returns current process RAM usage in MB"""
    return PROCESS.memory_info().rss / (1024 ** 2)

```

```

[7]: # =====
# 1 SUPPORT VECTOR MACHINE (LINEAR + RBF)
# =====
# This block trains an SVM classifier using GridSearchCV
# and performs post-training feature analysis including:
# - Feature importance (linear kernel only)
# - t-SNE visualization
# - UMAP visualization
# =====

print("\n-----")
print(" Training Support Vector Machine...")
print("-----\n")

# -----
# Step 1: Define SVM pipeline
# -----
# StandardScaler is REQUIRED for SVM to:
# - normalize feature magnitudes
# - ensure stable optimization
# - improve convergence
# -----

svm_pipeline = Pipeline(
    steps=[
        ("scaler", StandardScaler()),
        (
            "svm",
            SVC(
                probability=True, # Required for ROC/AUC computation
                random_state=42
            ),
        ),
    ]
)

# -----
# Step 2: Define hyperparameter search space
# -----
# - Linear kernel → supports feature importance via coefficients

```

```

# - RBF kernel → non-linear, higher capacity but no interpretability
# -----

svm_param_grid = [
    {
        "svm__kernel": ["linear"],
        "svm__C": [0.1, 1, 10],
    },
    {
        "svm__kernel": ["rbf"],
        "svm__C": [0.1, 1, 10],
        "svm__gamma": [0.01, 0.1, 1],
    },
]

# -----
# Step 3: GridSearchCV setup
# -----
# - Cross-validation ensures robust hyperparameter selection
# - Accuracy is used as optimization metric
# -----

svm_grid = GridSearchCV(
    estimator=svm_pipeline,
    param_grid=svm_param_grid,
    cv=CV_FOLDS,
    scoring="accuracy",
    n_jobs=-1,
    verbose=1,
)

# -----
# Step 4: Measure training time & memory
# -----

train_start_ram = get_ram_mb()
train_start_time = time.perf_counter()

svm_grid.fit(X_train, y_train)

train_end_time = time.perf_counter()
train_end_ram = get_ram_mb()

svm_training_time = train_end_time - train_start_time
svm_training_memory_mb = train_end_ram - train_start_ram

print(f"\n* SVM Training Time: {svm_training_time:.4f} seconds")

```

```

print(f"* SVM Training Memory Usage: {svm_training_memory_mb:.2f} MB")
print(f"* Best SVM Parameters:", svm_grid.best_params_)

# -----
# Step 5: Extract best trained model
# -----
# best_svm is a FULL pipeline:
# scaler → trained SVM
# -----

best_svm = svm_grid.best_estimator_

# =====
# MODEL SIZE (SVM)
# =====
# Definition used in report:
# - Linear SVM → number of learned coefficients
# - RBF SVM → number of support vectors
# =====

svm_model = best_svm.named_steps["svm"]

if svm_model.kernel == "linear":
    svm_model_size = svm_model.coef_.size
    model_size_desc = "Number of learned coefficients"
else:
    svm_model_size = svm_model.support_vectors_.shape[0]
    model_size_desc = "Number of support vectors"

print(f" SVM Model Size ({model_size_desc}): {svm_model_size}")

# =====
# CLASSICAL FEATURE ANALYSIS (POST-TRAINING)
# =====
# IMPORTANT:
# - Feature importance requires trained model
# - Feature space visualization must use SCALED features
# =====

# -----
# Step 6: Feature importance (Linear SVM only)
# -----
# Only linear SVM exposes coefficients (coef_)
# -----

feature_names = get_classical_feature_names(
    hist_bins=16,

```

```

        lbp_points=8,
        include_motion=True
    )

print(f"\n Total classical features: {len(feature_names)}\n")

if best_svm.named_steps["svm"].kernel == "linear":
    visualize_classical_feature_importance(
        model=best_svm.named_steps["svm"],      # trained linear SVM
        feature_names=feature_names,            # handcrafted feature names
        top_k=20,
        save_name="svm_feature_importance.png"
    )
else:
    print("\n Feature importance skipped (RBF kernel selected)")

# -----
# Step 7: Extract scaled features for visualization
# -----
# Feature space must match what the SVM actually learned
# -----

X_train_scaled = best_svm.named_steps["scaler"].transform(X_train)

# -----
# Step 8: t-SNE visualization
# -----
# Shows how samples cluster in reduced 2D space
# -----

visualize_classical_feature_space(
    features=X_train_scaled,
    labels=y_train,
    method="tsne",
    save_name="svm_tsne.png"
)

# -----
# Step 9: UMAP visualization
# -----
# Preserves both local and global structure
# -----

visualize_classical_feature_space(
    features=X_train_scaled,
    labels=y_train,
    method="umap",

```

```

    save_name="svm_umap.png"
)

# =====
#  INFERENCE TIME & MEMORY PER VIDEO
# =====

infer_start_ram = get_ram_mb()
inference_start_time = time.perf_counter()

y_pred = best_svm.predict(X_test)

inference_end_time = time.perf_counter()
infer_end_ram = get_ram_mb()

svm_inference_time_per_video = (
    (inference_end_time - inference_start_time) / len(X_test)
)

svm_inference_memory_mb = infer_end_ram - infer_start_ram

print(
    f" Avg Inference Time per Video: "
    f"{svm_inference_time_per_video:.6f} seconds"
)
print(
    f" Inference Memory Usage: "
    f"{svm_inference_memory_mb:.2f} MB\n"
)

# =====
#  ROC / AUC PROBABILITIES
# =====
# probability=True enables predict_proba()
# =====

y_scores = best_svm.predict_proba(X_test)

# =====
#  UNIFIED EVALUATION
# =====

class_names = [
    LABEL_TO_CLASS[i].replace("class_", "")
    for i in range(len(LABEL_TO_CLASS))
]

```

```

svm_metrics = evaluate_classification(
    y_true=y_test,
    y_pred=y_pred,
    y_scores=y_scores,
    class_names=class_names,
    model_name="svm"
)

# -----
# Store computational metrics for comparative analysis
# -----

svm_metrics["training_time"] = svm_training_time
svm_metrics["inference_time_per_video"] = svm_inference_time_per_video
svm_metrics["training_memory_mb"] = svm_training_memory_mb
svm_metrics["inference_memory_mb"] = svm_inference_memory_mb
svm_metrics["model_size"] = svm_model_size
svm_metrics["model_size_definition"] = model_size_desc

print("\n SVM Evaluation Completed")

```

Training Support Vector Machine..

Fitting 5 folds for each of 12 candidates, totalling 60 fits

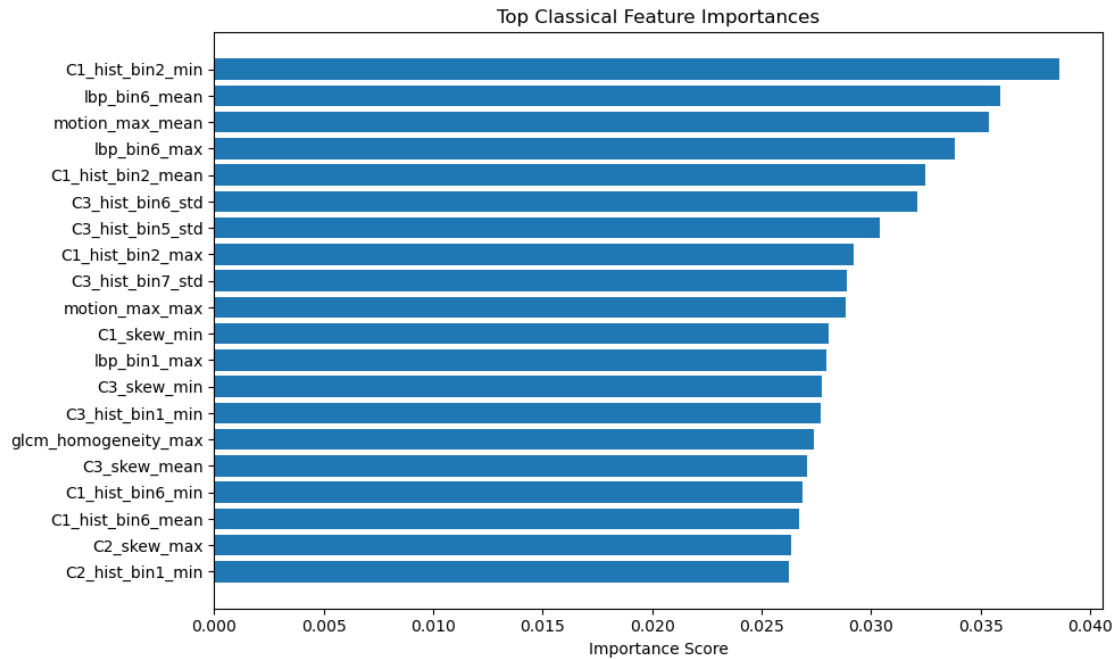
```

* SVM Training Time: 0.6031 seconds
* SVM Training Memory Usage: 0.00 MB
* Best SVM Parameters: {'svm__C': 0.1, 'svm__kernel': 'linear'}
  SVM Model Size (Number of learned coefficients): 3990

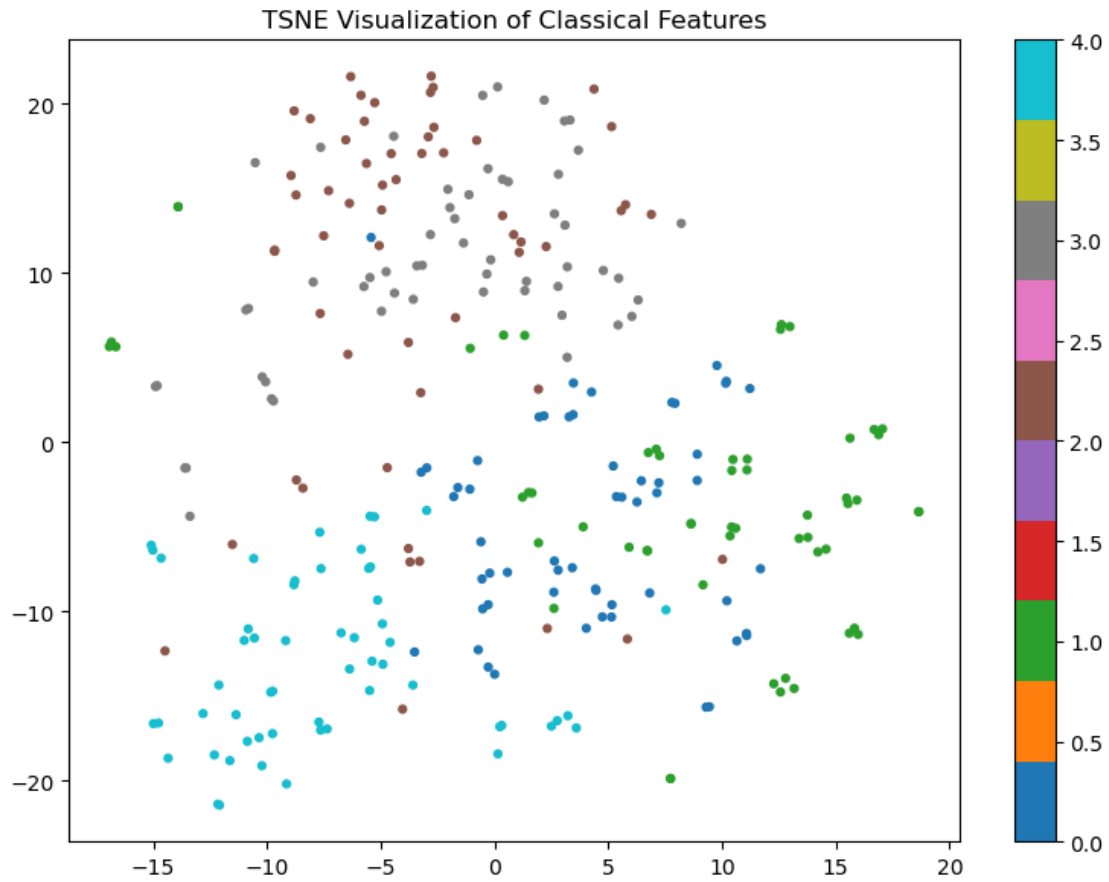
```

Total classical features: 399

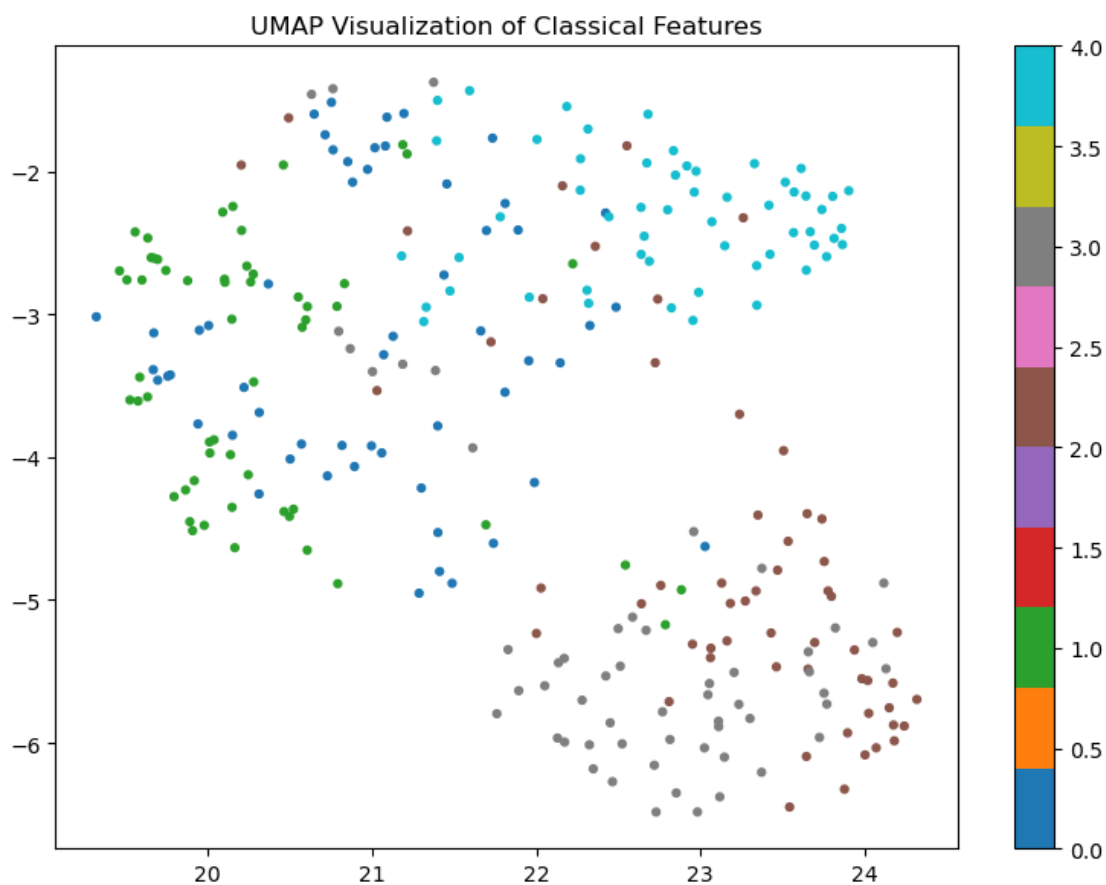
[Saved] Feature importance → /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classical/svm_feature_importance.png



[Saved] TSNE plot → /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classical/svm_tsne.png



[Saved] UMAP plot → /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classical/svm_umap.png



Avg Inference Time per Video: 0.000025 seconds

Inference Memory Usage: 0.00 MB

Starting evaluation for: svm

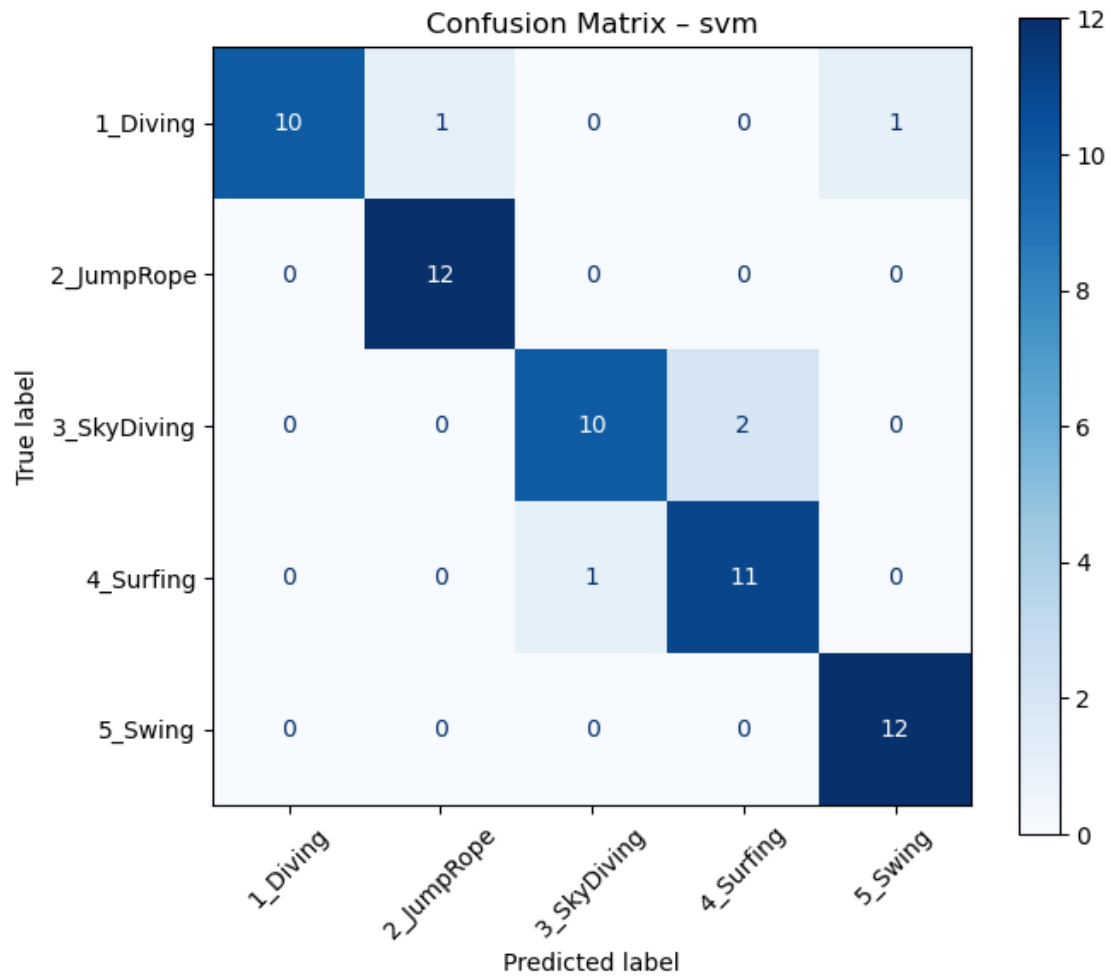
Overall Performance Metrics

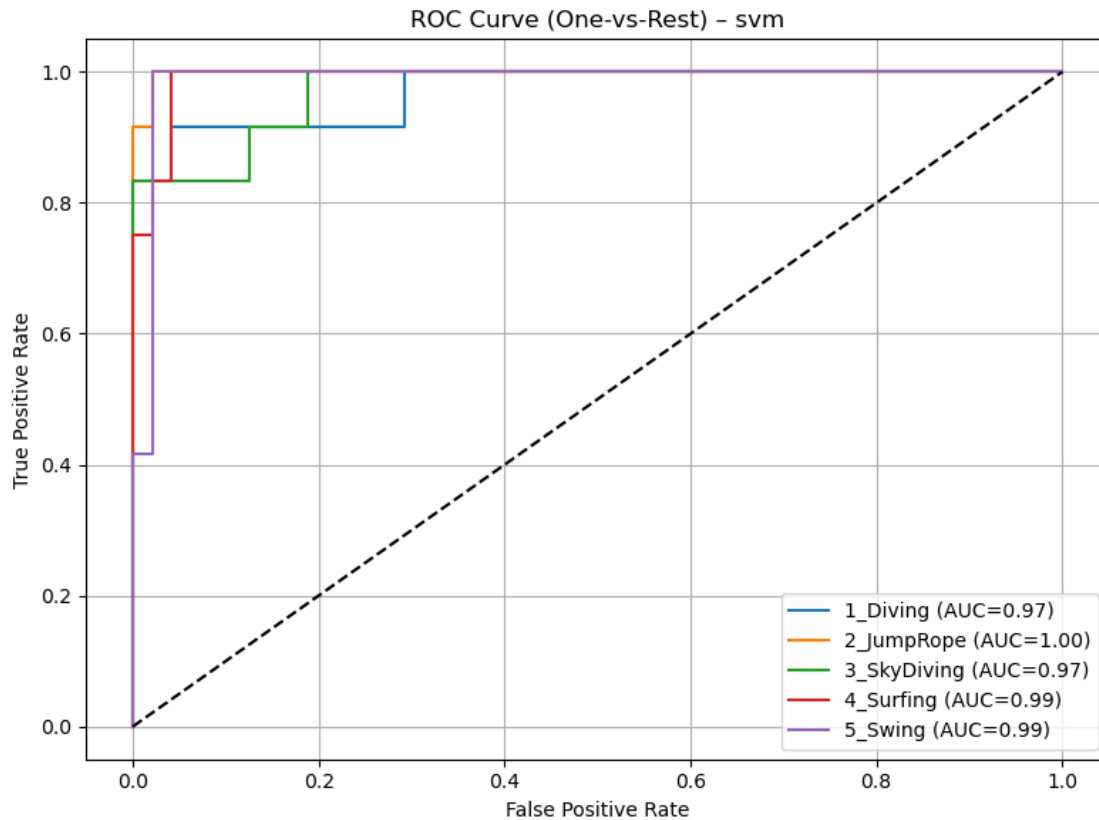
	Metric	Value
0	Accuracy	0.9167
1	Macro Precision	0.9203
2	Macro Recall	0.9167
3	Macro F1-Score	0.9157

Per-Class Classification Report

	precision	recall	f1-score	support
1_Diving	1.000000	0.833333	0.909091	12.000000
2_JumpRope	0.923077	1.000000	0.960000	12.000000
3_SkyDiving	0.909091	0.833333	0.869565	12.000000

4_Surfing	0.846154	0.916667	0.880000	12.000000
5_Swing	0.923077	1.000000	0.960000	12.000000
accuracy	0.916667	0.916667	0.916667	0.916667
macro avg	0.920280	0.916667	0.915731	60.000000
weighted avg	0.920280	0.916667	0.915731	60.000000





Evaluation completed successfully

Model Performance Interpretation

- The svm model achieves an overall accuracy of 91.67%, indicating strong classification performance across video categories.
- Macro-averaged precision, recall, and F1-score are closely aligned, suggesting balanced predictive behavior across classes.
- The similarity between macro and weighted averages confirms a balanced dataset with uniform class representation.
- The highest class-wise performance is observed for '2_JumpRope', while most misclassifications occur in '1_Diving'.
- Errors are distributed across multiple classes, suggesting scope for improving feature discrimination.

SVM Evaluation Completed

```
[9]: # =====
# 2 RANDOM FOREST CLASSIFIER
# =====
# This block trains a Random Forest classifier and performs
```

```

# post-training feature analysis including:
# - Feature importance (tree-based)
# - t-SNE visualization
# - UMAP visualization
# =====

print("\n-----")
print(" Training Random Forest...")
print("-----\n")

# -----
# Step 1: Initialize Random Forest
# -----
# Random Forest:
# - Does NOT require feature scaling
# - Learns feature importance via split criteria
# - Robust to noise and overfitting
# -----

rf = RandomForestClassifier(
    random_state=RANDOM_STATE,
    n_jobs=-1,
)

# -----
# Step 2: Hyperparameter grid
# -----
# Controls:
# - number of trees
# - tree depth
# - split granularity
# -----

rf_param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [10, 20, None],
    "min_samples_split": [2, 5],
}

# -----
# Step 3: GridSearchCV
# -----
# Cross-validation ensures stable hyperparameter selection
# -----

rf_grid = GridSearchCV(
    estimator=rf,

```

```

    param_grid=rf_param_grid,
    cv=CV_FOLDS,
    scoring="accuracy",
    n_jobs=-1,
    verbose=1,
)

# -----
#   Step 4: Measure training time & memory
# -----

train_start_ram = get_ram_mb()
train_start_time = time.perf_counter()

rf_grid.fit(X_train, y_train)

train_end_time = time.perf_counter()
train_end_ram = get_ram_mb()

rf_training_time = train_end_time - train_start_time
rf_training_memory_mb = train_end_ram - train_start_ram

print(f"\n RF Training Time: {rf_training_time:.4f} seconds")
print(f" RF Training Memory Usage: {rf_training_memory_mb:.2f} MB")
print(f"\n Best RF Parameters: {rf_grid.best_params_}\n")

# -----
#   Step 5: Extract best trained model
# -----

best_rf = rf_grid.best_estimator_

# =====
#   MODEL SIZE (RANDOM FOREST)
# =====
# Definition used in report:
# - Model size = total number of decision tree nodes
# - Reflects storage + decision complexity
# =====

rf_model_size = sum(
    tree.tree_.node_count for tree in best_rf.estimators_
)

print(f" Random Forest Model Size (Total Tree Nodes): {rf_model_size}")

# =====

```

```

# CLASSICAL FEATURE ANALYSIS (POST-TRAINING)
# =====
# IMPORTANT:
# - Random Forest ALWAYS supports feature_importances_
# - Feature space visualization uses RAW features
#   (no scaling required for tree models)
# =====

# -----
# Step 6: Feature importance (Tree-based)
# -----

feature_names = get_classical_feature_names(
    hist_bins=16,
    lbp_points=8,
    include_motion=True
)

visualize_classical_feature_importance(
    model=best_rf,
    feature_names=feature_names,
    top_k=20,
    save_name="rf_feature_importance.png"
)

# -----
# Step 7: Feature space visualization (t-SNE)
# -----
# Raw handcrafted features are sufficient for RF
# -----

visualize_classical_feature_space(
    features=X_train,
    labels=y_train,
    method="tsne",
    save_name="rf_tsne.png"
)

# -----
# Step 8: Feature space visualization (UMAP)
# -----

visualize_classical_feature_space(
    features=X_train,
    labels=y_train,
    method="umap",
    save_name="rf_umap.png"
)

```

```

)

# =====
#  INFERENCE TIME & MEMORY PER VIDEO
# =====
# Measures average prediction latency
# =====

infer_start_ram = get_ram_mb()
inference_start_time = time.perf_counter()

y_pred = best_rf.predict(X_test)

inference_end_time = time.perf_counter()
infer_end_ram = get_ram_mb()

rf_inference_time_per_video = (
    (inference_end_time - inference_start_time) / len(X_test)
)

rf_inference_memory_mb = infer_end_ram - infer_start_ram

print(
    f"\n Avg Inference Time per Video: "
    f"{rf_inference_time_per_video:.6f} seconds"
)
print(
    f" RF Inference Memory Usage: "
    f"{rf_inference_memory_mb:.2f} MB\n"
)

# =====
#  ROC / AUC PROBABILITIES
# =====
# Random Forest natively supports predict_proba()
# =====

y_scores = best_rf.predict_proba(X_test)

# =====
#  UNIFIED EVALUATION
# =====

class_names = [
    LABEL_TO_CLASS[i].replace("class_", "")
    for i in range(len(LABEL_TO_CLASS))
]

```

```

rf_metrics = evaluate_classification(
    y_true=y_test,
    y_pred=y_pred,
    y_scores=y_scores,
    class_names=class_names,
    model_name="random_forest"
)

# -----
# Store computational efficiency metrics
# -----

rf_metrics["training_time"] = rf_training_time
rf_metrics["inference_time_per_video"] = rf_inference_time_per_video
rf_metrics["training_memory_mb"] = rf_training_memory_mb
rf_metrics["inference_memory_mb"] = rf_inference_memory_mb
rf_metrics["model_size"] = rf_model_size
rf_metrics["model_size_definition"] = "Total number of tree nodes"

print("\n Random Forest Evaluation Completed")

```

```

-----
Training Random Forest...
-----

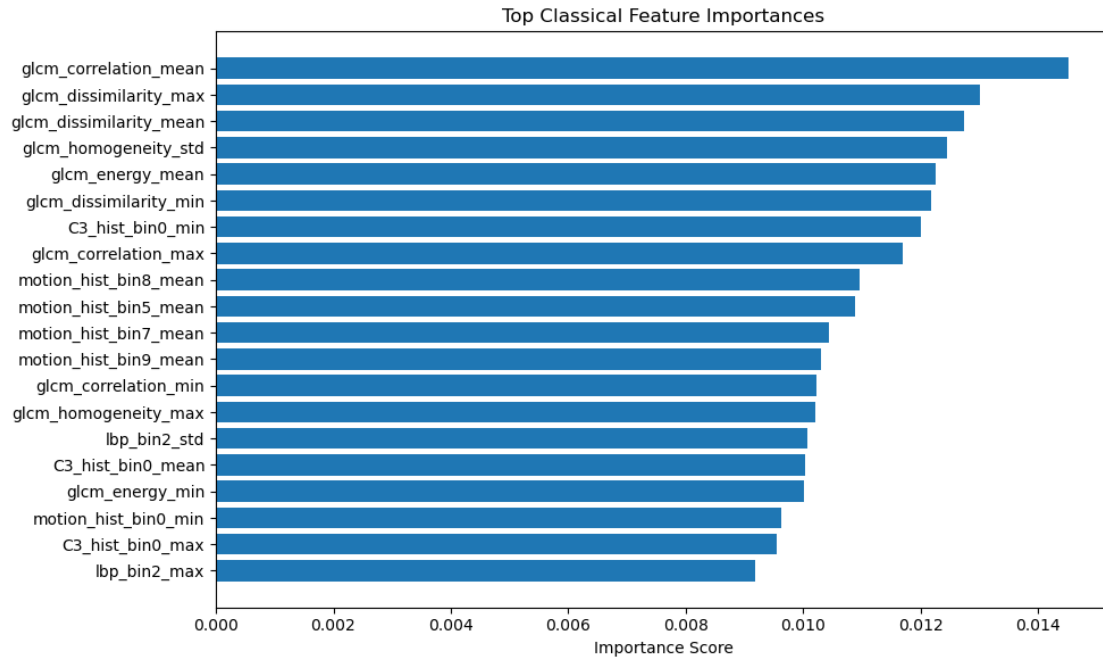
```

Fitting 5 folds for each of 12 candidates, totalling 60 fits

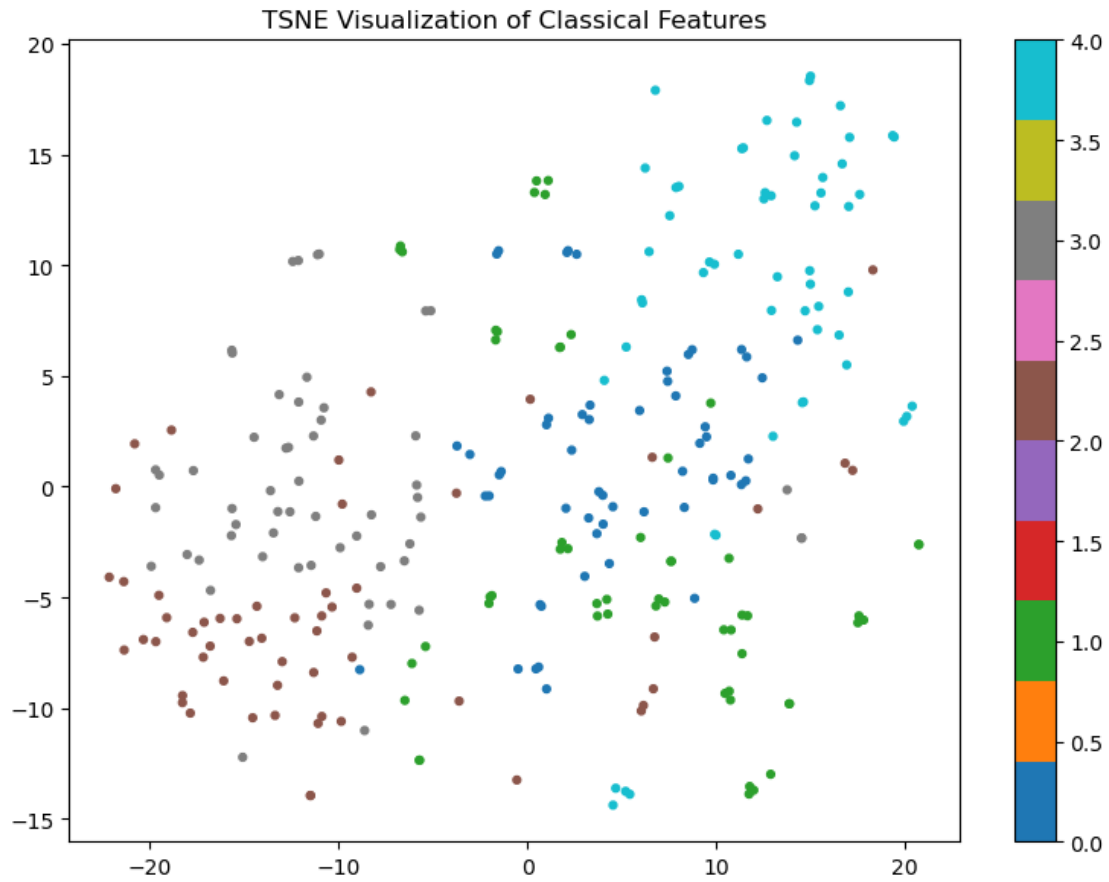
RF Training Time: 6.6075 seconds
RF Training Memory Usage: 1.12 MB

Best RF Parameters: {'max_depth': 10, 'min_samples_split': 2, 'n_estimators': 200}

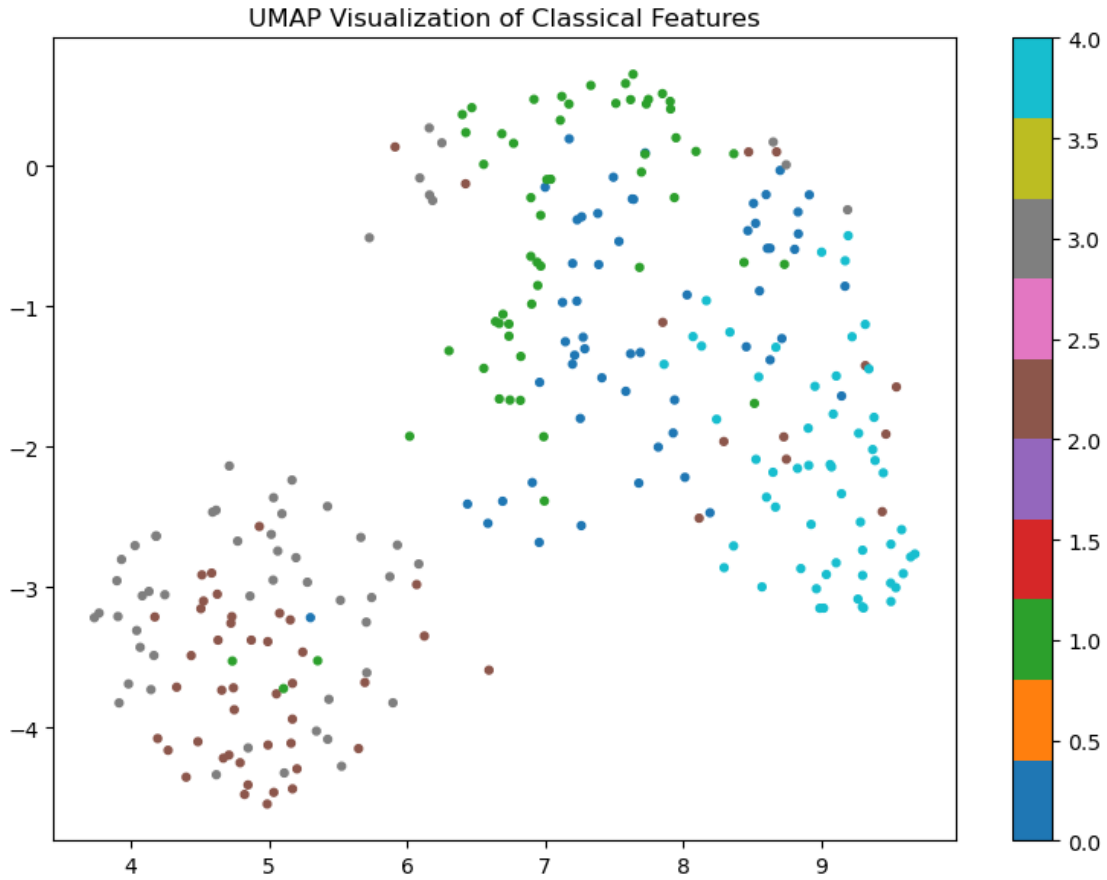
Random Forest Model Size (Total Tree Nodes): 12564
[Saved] Feature importance → /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classical/rf_feature_importance.png



[Saved] TSNE plot → /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classical/rf_tsne.png



[Saved] UMAP plot → /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classical/rf_umap.png



Avg Inference Time per Video: 0.000598 seconds

RF Inference Memory Usage: 0.00 MB

Starting evaluation for: random_forest

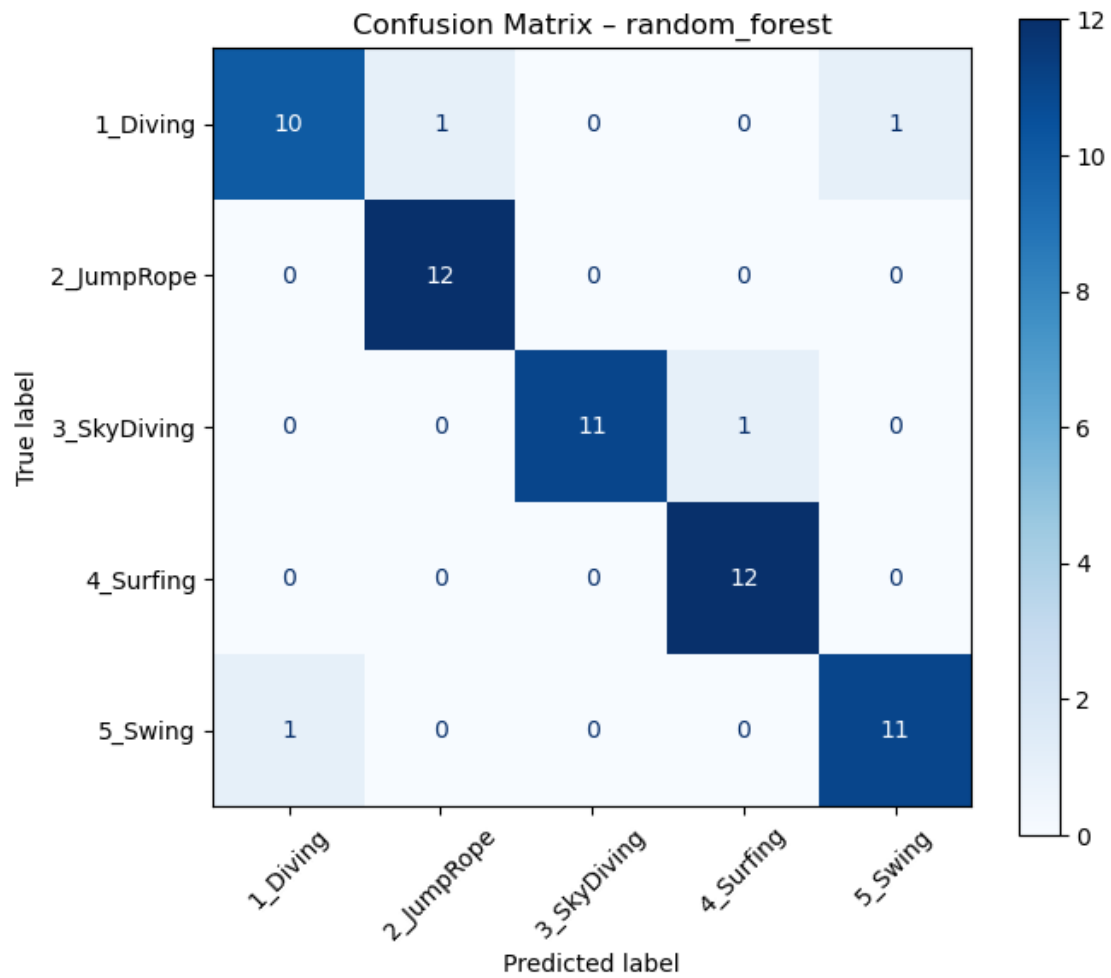
Overall Performance Metrics

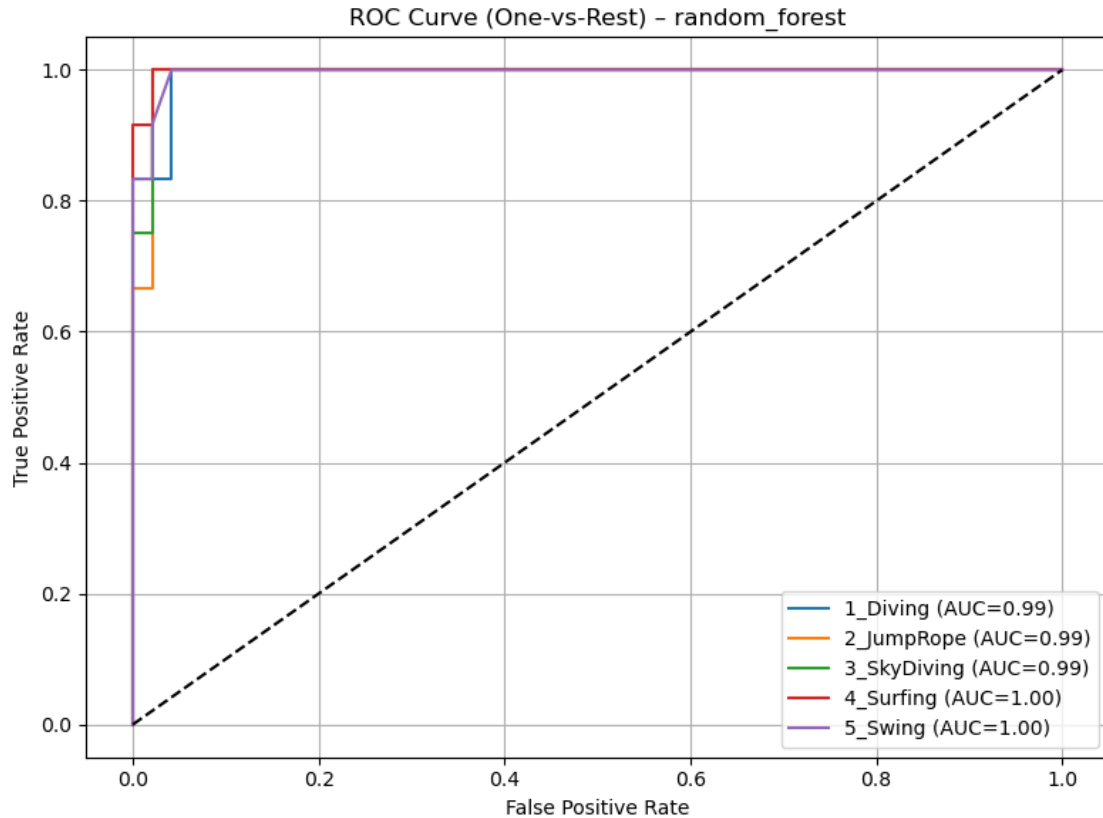
	Metric	Value
0	Accuracy	0.9333
1	Macro Precision	0.9344
2	Macro Recall	0.9333
3	Macro F1-Score	0.9326

Per-Class Classification Report

	precision	recall	f1-score	support
1_Diving	0.909091	0.833333	0.869565	12.000000
2_JumpRope	0.923077	1.000000	0.960000	12.000000

3_SkyDiving	1.000000	0.916667	0.956522	12.000000
4_Surfing	0.923077	1.000000	0.960000	12.000000
5_Swing	0.916667	0.916667	0.916667	12.000000
accuracy	0.933333	0.933333	0.933333	0.933333
macro avg	0.934382	0.933333	0.932551	60.000000
weighted avg	0.934382	0.933333	0.932551	60.000000





Evaluation completed successfully

Model Performance Interpretation

- The random_forest model achieves an overall accuracy of 93.33%, indicating strong classification performance across video categories.
- Macro-averaged precision, recall, and F1-score are closely aligned, suggesting balanced predictive behavior across classes.
- The similarity between macro and weighted averages confirms a balanced dataset with uniform class representation.
- The highest class-wise performance is observed for '2_JumpRope', while most misclassifications occur in '1_Diving'.
- Errors are distributed across multiple classes, suggesting scope for improving feature discrimination.

Random Forest Evaluation Completed

```
[10]: # =====
# 3 k-NEAREST NEIGHBORS (k-NN)
# =====
# This block trains a k-NN classifier and performs
```

```

# post-training feature analysis including:
# - t-SNE visualization
# - UMAP visualization
#
# NOTE:
# k-NN is a non-parametric, instance-based learner.
# It does NOT learn explicit feature weights.
# Therefore, feature importance is NOT applicable.
# =====

print("\n-----")
print("  Training k-Nearest Neighbors...")
print("-----\n")

# -----
#   Step 1: Define k-NN pipeline
# -----
# Feature scaling is CRITICAL for k-NN because:
# - Distance metrics are scale-sensitive
# - Unscaled features bias nearest neighbors
# -----

knn_pipeline = Pipeline(
    steps=[
        ("scaler", StandardScaler()),
        (
            "knn",
            KNeighborsClassifier(
                weights="distance"    # improves probability estimates
            ),
        ),
    ]
)

# -----
#   Step 2: Hyperparameter grid
# -----
# - n_neighbors controls local vs global behavior
# - metric defines distance computation
# -----

knn_param_grid = {
    "knn__n_neighbors": [3, 5, 7, 9],
    "knn__metric": ["euclidean", "manhattan"],
}

# -----

```

```

# Step 3: GridSearchCV
# -----
# Cross-validation ensures robust selection of k
# -----

knn_grid = GridSearchCV(
    estimator=knn_pipeline,
    param_grid=knn_param_grid,
    cv=CV_FOLDS,
    scoring="accuracy",
    n_jobs=-1,
    verbose=1,
)

# -----
# Step 4: Measure training time & memory
# -----
# k-NN has minimal "training" time (lazy learner)
# -----

train_start_ram = get_ram_mb()
train_start_time = time.perf_counter()

knn_grid.fit(X_train, y_train)

train_end_time = time.perf_counter()
train_end_ram = get_ram_mb()

knn_training_time = train_end_time - train_start_time
knn_training_memory_mb = train_end_ram - train_start_ram

print(f"\n k-NN Training Time: {knn_training_time:.4f} seconds")
print(f" k-NN Training Memory Usage: {knn_training_memory_mb:.2f} MB")
print(f" Best k-NN Parameters: {knn_grid.best_params_} \n")

# -----
# Step 5: Extract best trained model
# -----
# best_knn is a full pipeline:
# scaler → k-NN
# -----

best_knn = knn_grid.best_estimator_

# =====
# MODEL SIZE (k-NN)
# =====

```

```

# Definition used in report:
# - Model size = number of stored training instances
# - Reflects memory + inference complexity
# =====

knn_model = best_knn.named_steps["knn"]
knn_model_size = knn_model._fit_X.shape[0]

print(f" k-NN Model Size (Stored Training Instances): {knn_model_size}")

# =====
#  FEATURE SPACE VISUALIZATION (POST-TRAINING)
# =====
# IMPORTANT:
# Feature space MUST match the scaled representation
# used internally by k-NN
# =====

# -----
#  Step 6: Extract scaled training features
# -----

X_train_scaled = best_knn.named_steps["scaler"].transform(X_train)

# -----
#  Step 7: t-SNE visualization
# -----

visualize_classical_feature_space(
    features=X_train_scaled,
    labels=y_train,
    method="tsne",
    save_name="knn_tsne.png"
)

# -----
#  Step 8: UMAP visualization
# -----

visualize_classical_feature_space(
    features=X_train_scaled,
    labels=y_train,
    method="umap",
    save_name="knn_umap.png"
)

# =====

```

```

# INFERENCE TIME & MEMORY PER VIDEO
# =====
# k-NN inference is slower due to distance computations
# =====

infer_start_ram = get_ram_mb()
inference_start_time = time.perf_counter()

y_pred = best_knn.predict(X_test)

inference_end_time = time.perf_counter()
infer_end_ram = get_ram_mb()

knn_inference_time_per_video = (
    (inference_end_time - inference_start_time) / len(X_test)
)

knn_inference_memory_mb = infer_end_ram - infer_start_ram

print(
    f" Avg Inference Time per Video: "
    f"{knn_inference_time_per_video:.6f} seconds"
)
print(
    f" k-NN Inference Memory Usage: "
    f"{knn_inference_memory_mb:.2f} MB\n"
)

# =====
# ROC / AUC PROBABILITIES
# =====
# weights="distance" enables predict_proba()
# =====

y_scores = best_knn.predict_proba(X_test)

# =====
# UNIFIED EVALUATION
# =====

class_names = [
    LABEL_TO_CLASS[i].replace("class_", "")
    for i in range(len(LABEL_TO_CLASS))
]

knn_metrics = evaluate_classification(
    y_true=y_test,

```

```

    y_pred=y_pred,
    y_scores=y_scores,
    class_names=class_names,
    model_name="knn",
)

# -----
# Store computational efficiency metrics
# -----

knn_metrics["training_time"] = knn_training_time
knn_metrics["inference_time_per_video"] = knn_inference_time_per_video
knn_metrics["training_memory_mb"] = knn_training_memory_mb
knn_metrics["inference_memory_mb"] = knn_inference_memory_mb
knn_metrics["model_size"] = knn_model_size
knn_metrics["model_size_definition"] = "Number of stored training instances"

print("\n k-NN Evaluation Completed")

```

```

-----
Training k-Nearest Neighbors...
-----

```

Fitting 5 folds for each of 8 candidates, totalling 40 fits

```

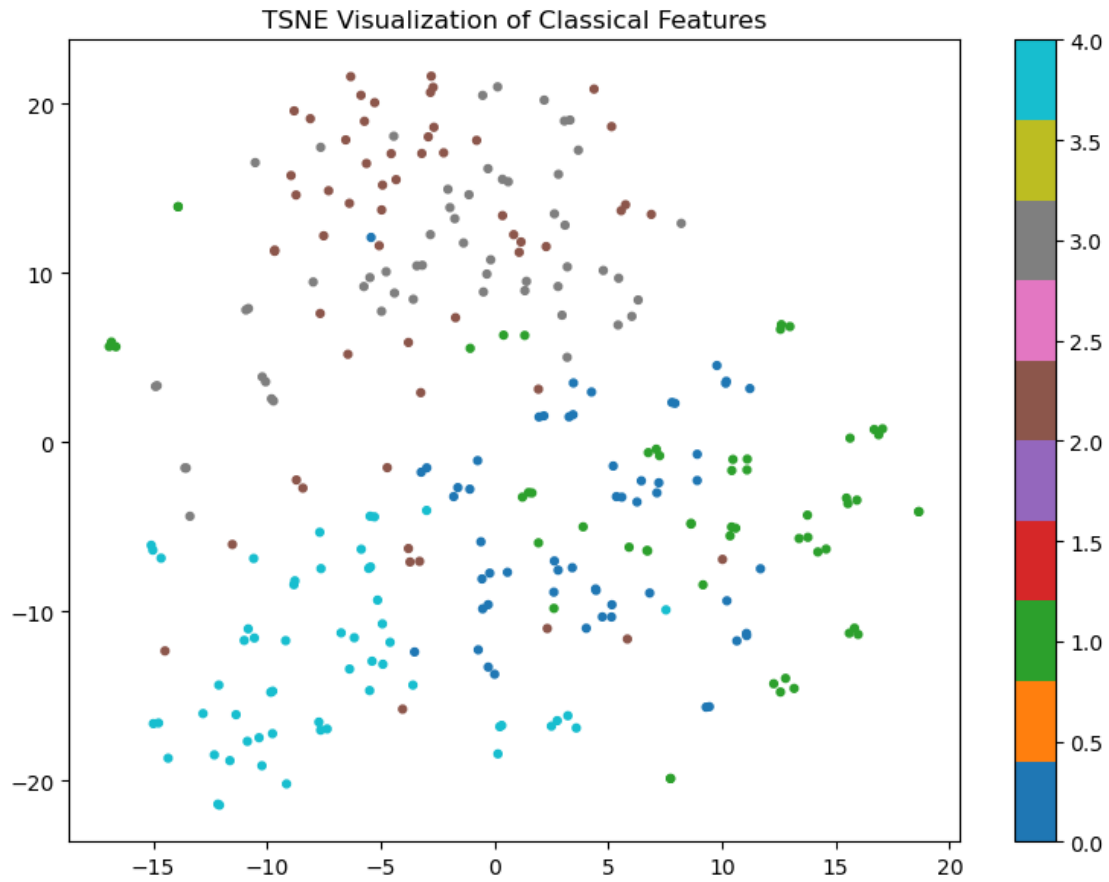
k-NN Training Time: 0.0810 seconds
k-NN Training Memory Usage: 0.00 MB
Best k-NN Parameters: {'knn__metric': 'manhattan', 'knn__n_neighbors': 7}

```

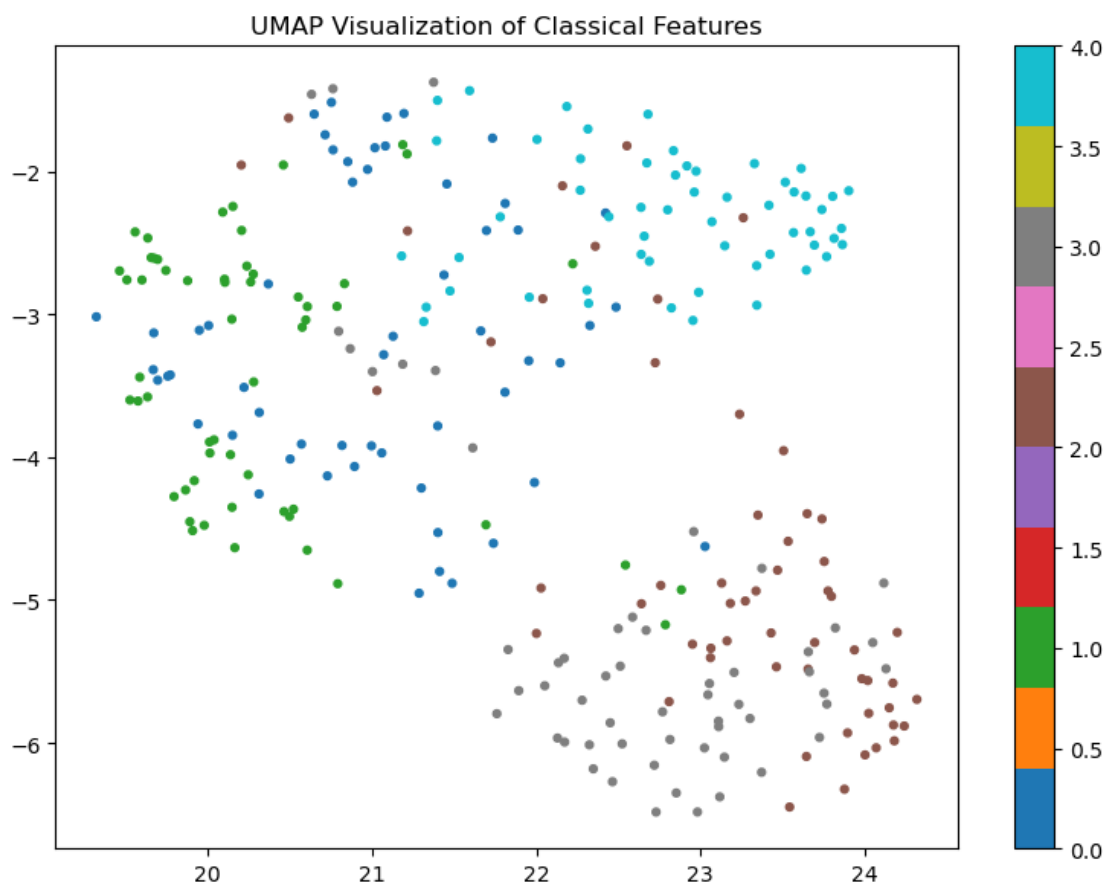
```

k-NN Model Size (Stored Training Instances): 280
[Saved] TSNE plot → /home/jovyan/va-assignment-
1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classi
cal/knn_tsne.png

```



[Saved] UMAP plot → /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/feature_visualizations/classical/knn_umap.png



Avg Inference Time per Video: 0.000082 seconds

k-NN Inference Memory Usage: 0.00 MB

Starting evaluation for: knn

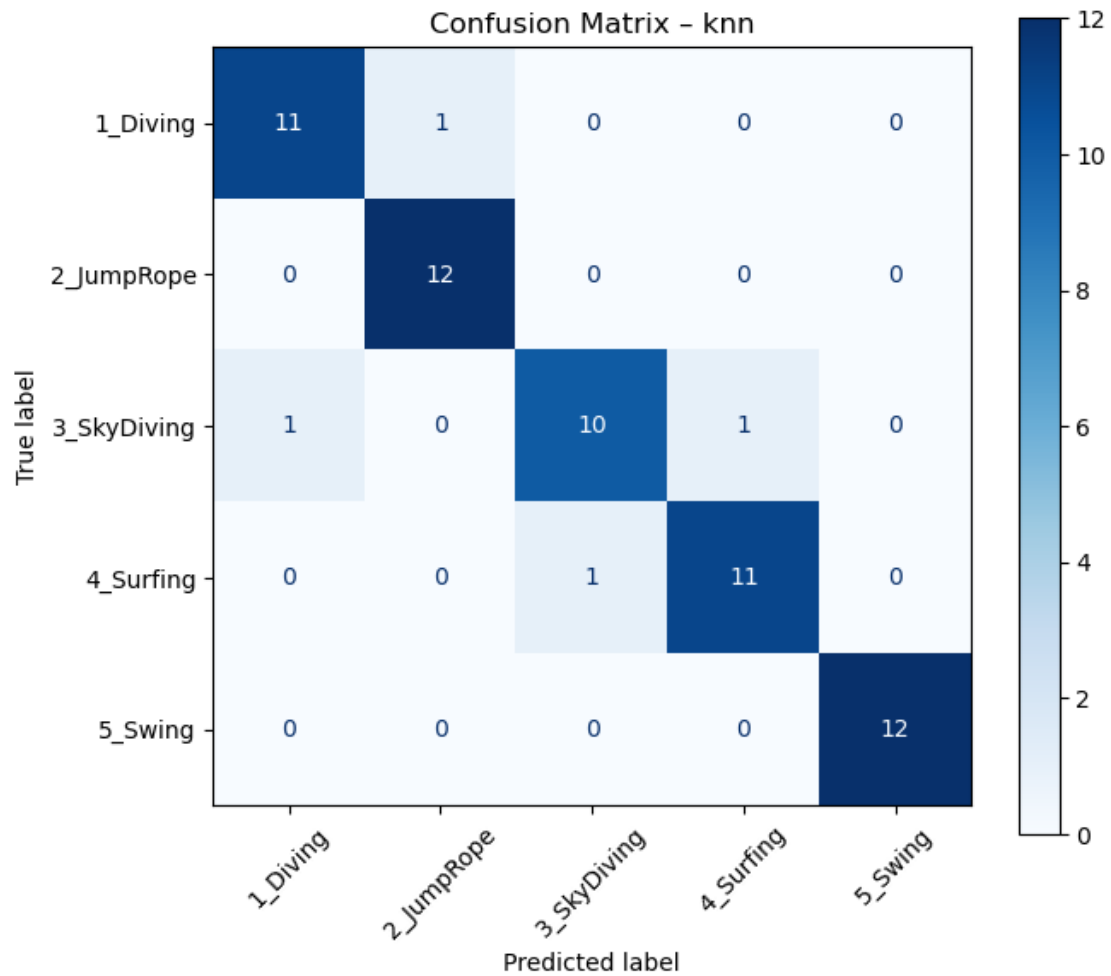
Overall Performance Metrics

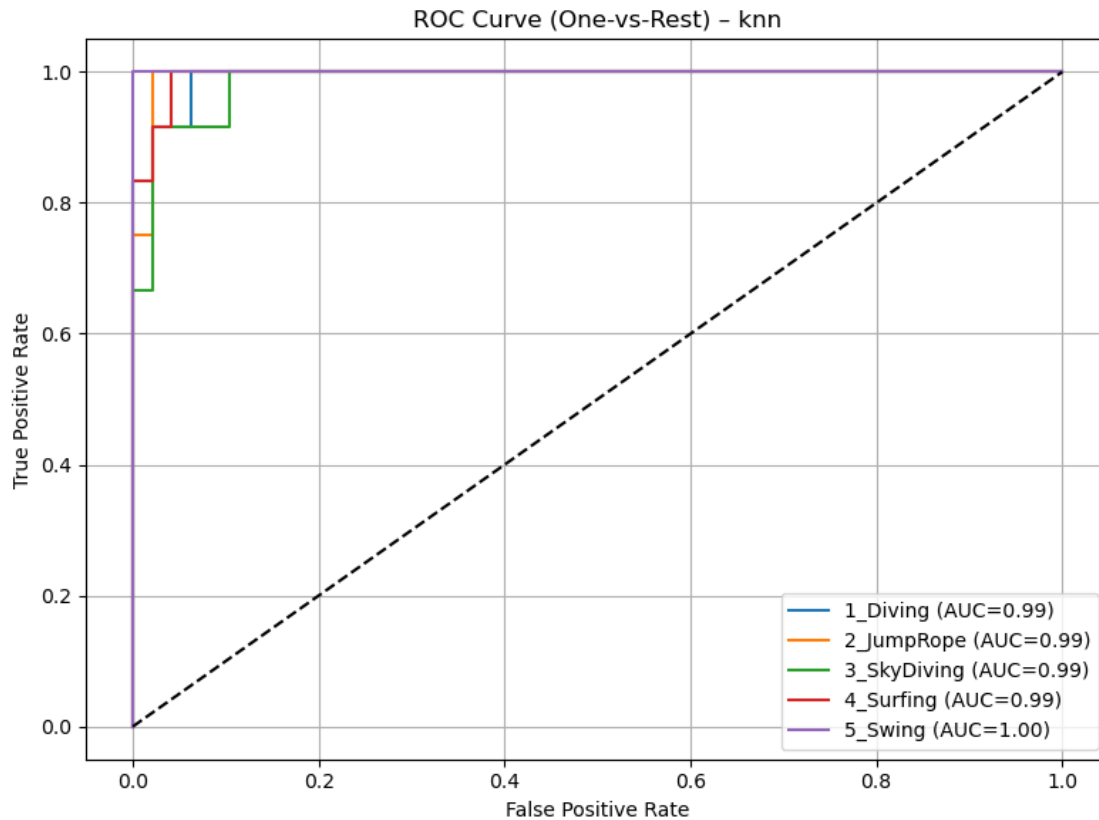
	Metric	Value
0	Accuracy	0.9333
1	Macro Precision	0.9331
2	Macro Recall	0.9333
3	Macro F1-Score	0.9326

Per-Class Classification Report

	precision	recall	f1-score	support
1_Diving	0.916667	0.916667	0.916667	12.000000
2_JumpRope	0.923077	1.000000	0.960000	12.000000
3_SkyDiving	0.909091	0.833333	0.869565	12.000000

4_Surfing	0.916667	0.916667	0.916667	12.000000
5_Swing	1.000000	1.000000	1.000000	12.000000
accuracy	0.933333	0.933333	0.933333	0.933333
macro avg	0.933100	0.933333	0.932580	60.000000
weighted avg	0.933100	0.933333	0.932580	60.000000





Evaluation completed successfully

Model Performance Interpretation

- The knn model achieves an overall accuracy of 93.33%, indicating strong classification performance across video categories.
- Macro-averaged precision, recall, and F1-score are closely aligned, suggesting balanced predictive behavior across classes.
- The similarity between macro and weighted averages confirms a balanced dataset with uniform class representation.
- The highest class-wise performance is observed for '5_Swing', while most misclassifications occur in '3_SkyDiving'.
- Errors are distributed across multiple classes, suggesting scope for improving feature discrimination.

k-NN Evaluation Completed

```
[21]: # =====
# CLASSICAL ML - FULL COMPARATIVE REPORT & COMPARATIVE ANALYSIS
# =====
```

```

import os
import tempfile
import joblib
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
from IPython.display import display, Markdown

# =====
# Utility: Compute Macro-Averaged ROC-AUC
# =====
def compute_macro_auc(roc_auc_dict):
    """
    Computes macro-average ROC-AUC
    from per-class AUC dictionary.
    """
    return sum(roc_auc_dict.values()) / len(roc_auc_dict)

# =====
# Utility: Measure serialized model memory usage
# =====
def get_model_memory_mb(model):
    """
    Estimates REAL memory footprint by:
    - Serializing the trained model to disk
    - Measuring file size in MB

    This reflects actual storage cost.
    """
    with tempfile.NamedTemporaryFile(delete=False) as tmp:
        joblib.dump(model, tmp.name)
        size_mb = os.path.getsize(tmp.name) / (1024 ** 2)
    os.remove(tmp.name)
    return size_mb

# =====
# Utility: Compute number of learned parameters
# =====
def compute_model_parameters(model, model_type):
    """
    Estimates model complexity as number of parameters.

    Model-specific logic:
    - SVM: support_vectors × feature_dim
    - Random Forest: total decision nodes across trees
    - k-NN: stored training samples × feature_dim
    """

```

```

# -----
# Handle SVM inside a pipeline
# -----
if model_type == "svm":
    if hasattr(model, "steps"): # pipeline
        svm_model = model.steps[-1][1]
    else:
        svm_model = model
    return svm_model.support_vectors_.shape[0] * svm_model.support_vectors_.
↪shape[1]

# -----
# Random Forest
# -----
elif model_type == "rf":
    return sum(tree.tree_.node_count for tree in model.estimators_)

# -----
# k-NN inside a pipeline
# -----
elif model_type == "knn":
    if hasattr(model, "steps"):
        knn_model = model.steps[-1][1]
    else:
        knn_model = model
    X_fit = knn_model._fit_X
    return X_fit.shape[0] * X_fit.shape[1]

else:
    return None

# =====
# Utility: Plotting function
# =====
def plot_metric_bar(df, metric, ylabel, title):
    """
    Generic bar plot utility with:
    - Clean labeling
    - Grid
    - Value annotations
    """
    plt.figure(figsize=(8, 5))
    bars = plt.bar(
        df["Model"],
        df[metric]
    )

```

```

plt.xlabel("Model")
plt.ylabel(ylabel)
plt.title(title)
plt.grid(axis="y")

for bar in bars:
    value = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value,
        f"{value:.3f}",
        ha="center",
        va="bottom"
    )

plt.tight_layout()
plt.show()

# =====
# 1 Generate Full Report Function
# =====
def generate_full_report(
    svm_metrics, rf_metrics, knn_metrics,
    best_svm, best_rf, best_knn, class_names
):
    """
    Generates a fully PDF-safe comparative report
    for classical ML models (SVM, Random Forest, k-NN).
    Includes:
    - Unified comparison table
    - Per-class metrics
    - Performance/efficiency/memory plots
    - Key observations
    - Dynamic final conclusion
    """

    # -----
    # Fix truncated text in Jupyter
    # -----
    pd.set_option('display.max_colwidth', None)

    # -----
    # Build unified comparison table
    # -----
    comparison_df = pd.DataFrame([

```

```

{
    "Model": "SVM",
    "Accuracy": svm_metrics["accuracy"],
    "Macro Precision": svm_metrics["precision_macro"],
    "Macro Recall": svm_metrics["recall_macro"],
    "Macro F1-Score": svm_metrics["f1_macro"],
    "Macro ROC-AUC": compute_macro_auc(svm_metrics["roc_auc"]),
    "Training Time (s)": svm_metrics["training_time"],
    "Inference Time / Sample (s)":_
    ↪svm_metrics["inference_time_per_video"],
    "Model Parameters": compute_model_parameters(best_svm, "svm"),
    "Memory Usage (MB)": get_model_memory_mb(best_svm),
},
{
    "Model": "Random Forest",
    "Accuracy": rf_metrics["accuracy"],
    "Macro Precision": rf_metrics["precision_macro"],
    "Macro Recall": rf_metrics["recall_macro"],
    "Macro F1-Score": rf_metrics["f1_macro"],
    "Macro ROC-AUC": compute_macro_auc(rf_metrics["roc_auc"]),
    "Training Time (s)": rf_metrics["training_time"],
    "Inference Time / Sample (s)":_
    ↪rf_metrics["inference_time_per_video"],
    "Model Parameters": compute_model_parameters(best_rf, "rf"),
    "Memory Usage (MB)": get_model_memory_mb(best_rf),
},
{
    "Model": "k-NN",
    "Accuracy": knn_metrics["accuracy"],
    "Macro Precision": knn_metrics["precision_macro"],
    "Macro Recall": knn_metrics["recall_macro"],
    "Macro F1-Score": knn_metrics["f1_macro"],
    "Macro ROC-AUC": compute_macro_auc(knn_metrics["roc_auc"]),
    "Training Time (s)": knn_metrics["training_time"],
    "Inference Time / Sample (s)":_
    ↪knn_metrics["inference_time_per_video"],
    "Model Parameters": compute_model_parameters(best_knn, "knn"),
    "Memory Usage (MB)": get_model_memory_mb(best_knn),
},
])
comparison_df = comparison_df.round(4)

print("\n UNIFIED MODEL COMPARISON TABLE")
display(comparison_df)

# -----
# Plots

```

```

# -----
plot_metric_bar(comparison_df, "Accuracy", "Accuracy", "Accuracy Comparison_
↳Across Models")
plot_metric_bar(comparison_df, "Macro F1-Score", "F1 Score", "Macro_
↳F1-Score Comparison Across Models")
plot_metric_bar(comparison_df, "Macro ROC-AUC", "ROC-AUC", "Macro ROC-AUC_
↳Comparison Across Models")
plot_metric_bar(comparison_df, "Training Time (s)", "Seconds", "Training_
↳Time Comparison")
plot_metric_bar(comparison_df, "Inference Time / Sample (s)", "Seconds",_
↳"Inference Time per Sample Comparison")
plot_metric_bar(comparison_df, "Model Parameters", "Count", "Model Size_
↳(Number of Parameters)")
plot_metric_bar(comparison_df, "Memory Usage (MB)", "MB", "Memory Footprint_
↳Comparison")

# -----
# Per-Class Metrics
# -----
models = {
    "SVM": svm_metrics,
    "Random Forest": rf_metrics,
    "k-NN": knn_metrics
}

for model_name, metrics in models.items():
    display(Markdown(f"### Per-Class Metrics - {model_name}"))
    class_df = pd.DataFrame(metrics["classification_report"]).T
    rows_order = (
        [c for c in class_names if c in class_df.index] +
        [r for r in ["accuracy", "macro avg", "weighted avg"] if r in_
↳class_df.index]
    )
    class_df = class_df.loc[rows_order]
    display(class_df.round(4))

# -----
# Key Observations
# -----
accuracy = {m: v["accuracy"] for m, v in models.items()}
macro_f1 = {m: v["f1_macro"] for m, v in models.items()}
macro_precision = {m: v["precision_macro"] for m, v in models.items()}
macro_recall = {m: v["recall_macro"] for m, v in models.items()}

best_acc_model = max(accuracy, key=accuracy.get)
best_f1_model = max(macro_f1, key=macro_f1.get)

```

```

observations_df = pd.DataFrame([
    ["Highest Accuracy", f"{best_acc_model} ({accuracy[best_acc_model]:.3f})"],
    ["Best Overall Balance (Macro F1-Score)", f"{best_f1_model} ({macro_f1[best_f1_model]:.3f})"],
    ["Precision-Recall Characteristics",
     f"Random Forest: Precision={macro_precision['Random Forest']:.3f}, "
     f"Recall={macro_recall['Random Forest']:.3f}"],
    ["Model Characteristics",
     "SVM: margin-based classifier\n"
     "Random Forest: ensemble-based and robust\n"
     "k-NN: instance-based and distance-sensitive"]
], columns=["Observation", "Details"])
display(Markdown("## Key Observations"))
display(observations_df)

# -----
# Final Conclusion
# -----

best_accuracy_model = comparison_df.loc[comparison_df["Accuracy"].idxmax(), "Model"]
best_precision_model = comparison_df.loc[comparison_df["Macro Precision"].idxmax(), "Model"]
best_recall_model = comparison_df.loc[comparison_df["Macro Recall"].idxmax(), "Model"]
best_f1_model = comparison_df.loc[comparison_df["Macro F1-Score"].idxmax(), "Model"]
best_auc_model = comparison_df.loc[comparison_df["Macro ROC-AUC"].idxmax(), "Model"]

display(Markdown(
    f"""
### Final Conclusion

The comparative evaluation of classical machine learning models
reveals distinct strengths across different performance criteria.

Based on macro-averaged evaluation:
- **Highest Accuracy:** {best_accuracy_model}
- **Best Precision (Macro):** {best_precision_model}
- **Best Recall (Macro):** {best_recall_model}
- **Best Overall Balance (Macro F1-Score):** {best_f1_model}
- **Best Discriminative Capability (Macro ROC-AUC):** {best_auc_model}

Among the evaluated models, **{best_f1_model}** demonstrates the most

```

balanced and consistent performance across classes. These results indicate that no single model dominates all metrics, and model selection should be guided by task-specific requirements.

```

"""
    ))

    return comparison_df  # return for CSV export if needed

# =====
# 2 Save Comparative CSV
# =====
def save_comparative_csv(df, save_path):
    """
    Saves the unified comparison table including memory
    and model size metrics to CSV.
    """
    os.makedirs(os.path.dirname(save_path), exist_ok=True)
    df.to_csv(save_path, index=False)
    print(f"\n Comparative CSV saved → {save_path}")

# =====
# 3 Usage
# =====
CODE_DIR = Path.cwd()
PROJECT_ROOT = CODE_DIR.parent

# Generate full report and get the comparison table
comparison_df = generate_full_report(
    svm_metrics, rf_metrics, knn_metrics,
    best_svm, best_rf, best_knn, class_names
)

# Save CSV
save_comparative_csv(comparison_df, f"{PROJECT_ROOT}/results/
↳comparative_analysis.csv")

print("\n Full Comparative Analysis Completed Successfully.")

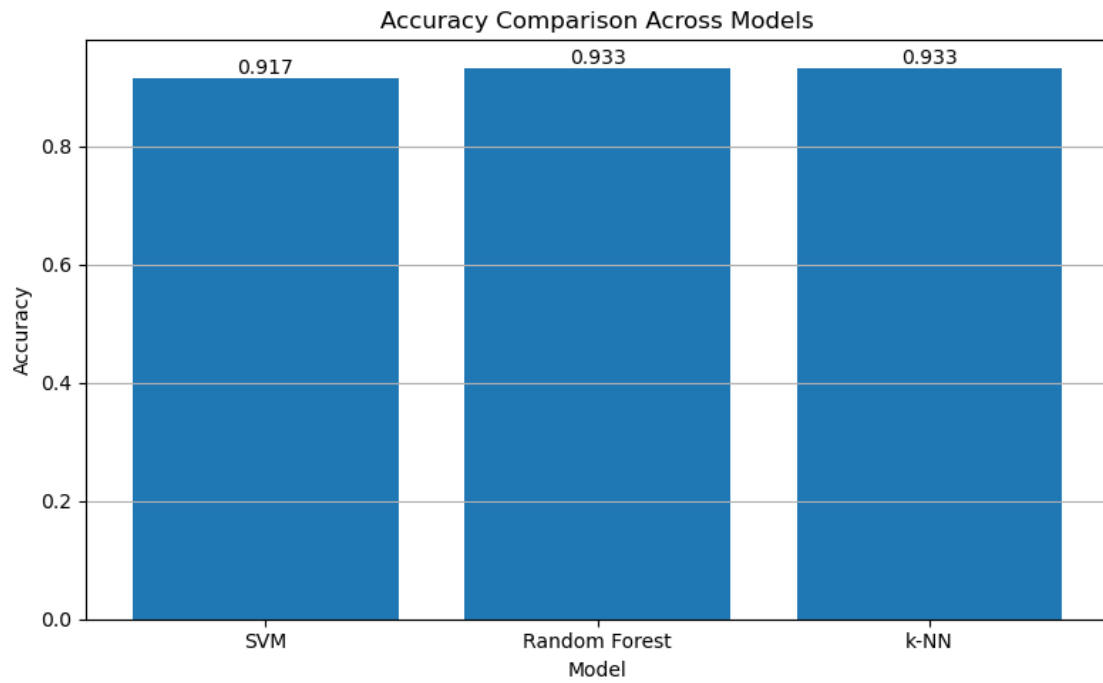
```

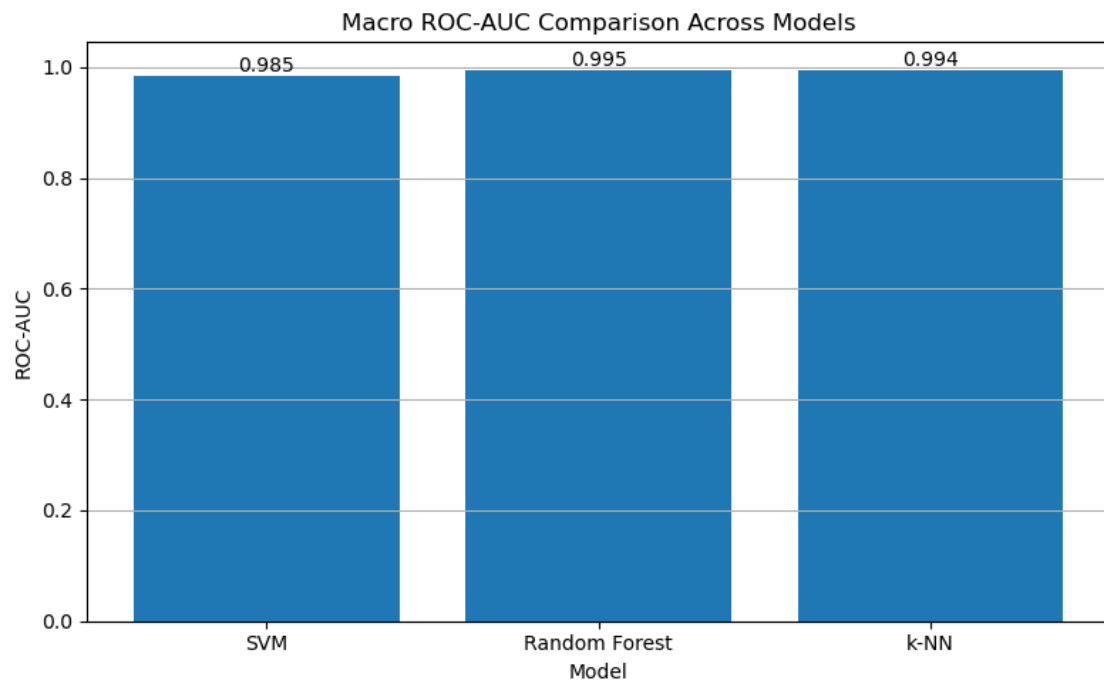
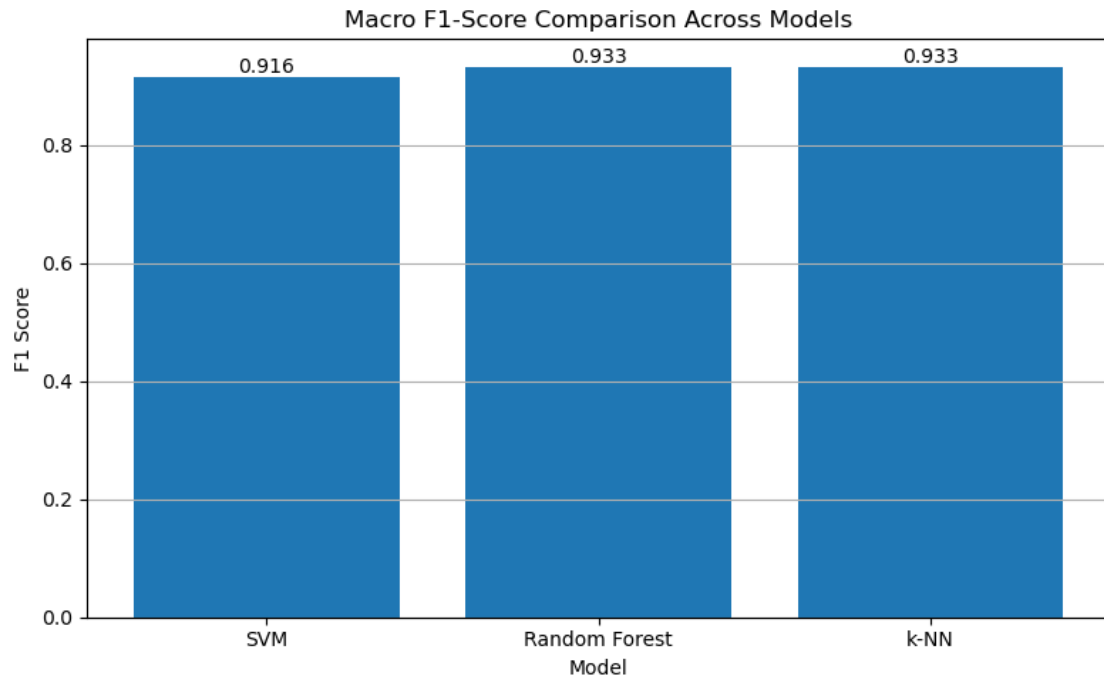
UNIFIED MODEL COMPARISON TABLE

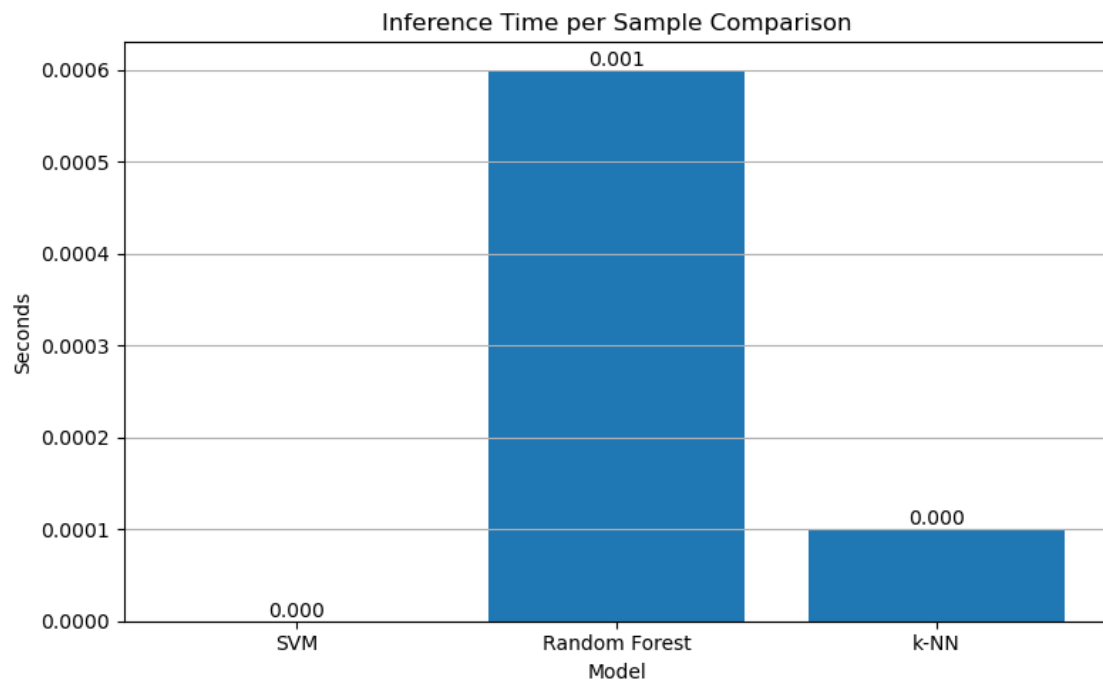
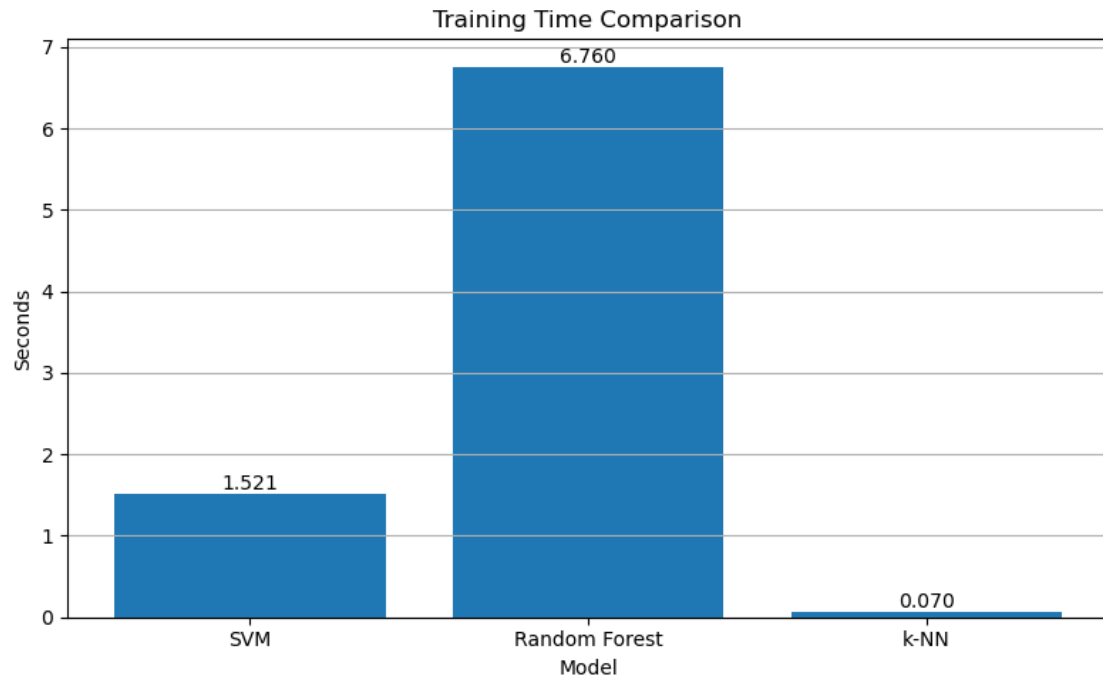
	Model	Accuracy	Macro Precision	Macro Recall	Macro F1-Score	\
0	SVM	0.9167	0.9203	0.9167	0.9157	
1	Random Forest	0.9333	0.9344	0.9333	0.9326	
2	k-NN	0.9333	0.9331	0.9333	0.9326	
	Macro ROC-AUC	Training Time (s)	Inference Time / Sample (s)			\
0	0.9847	1.5210	0.0000			

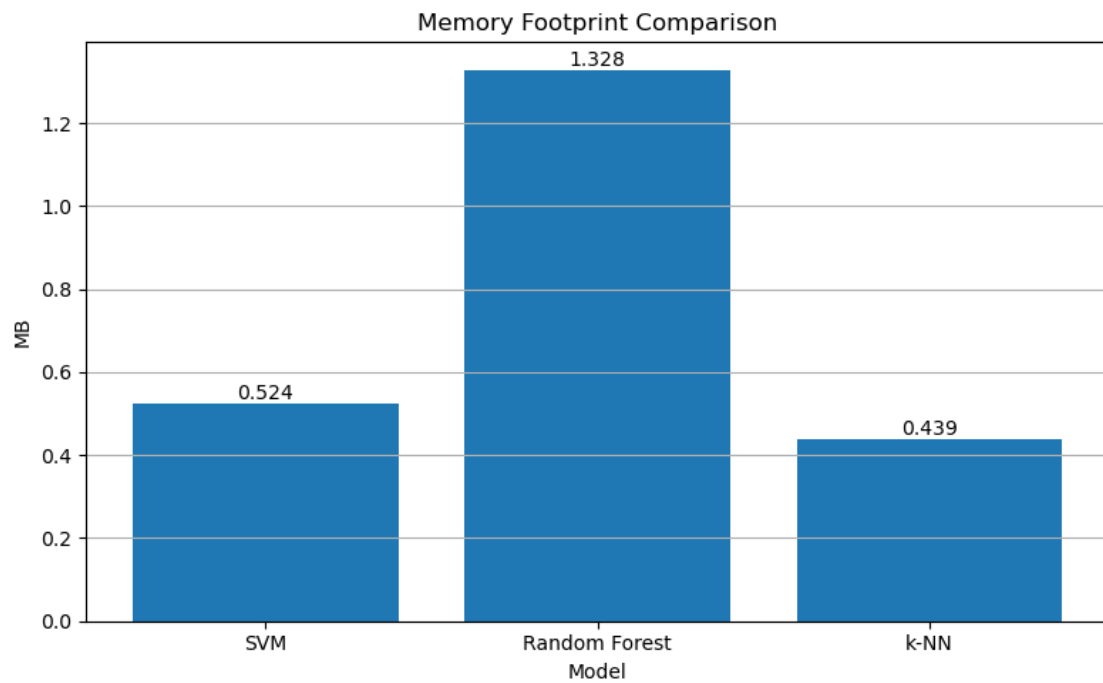
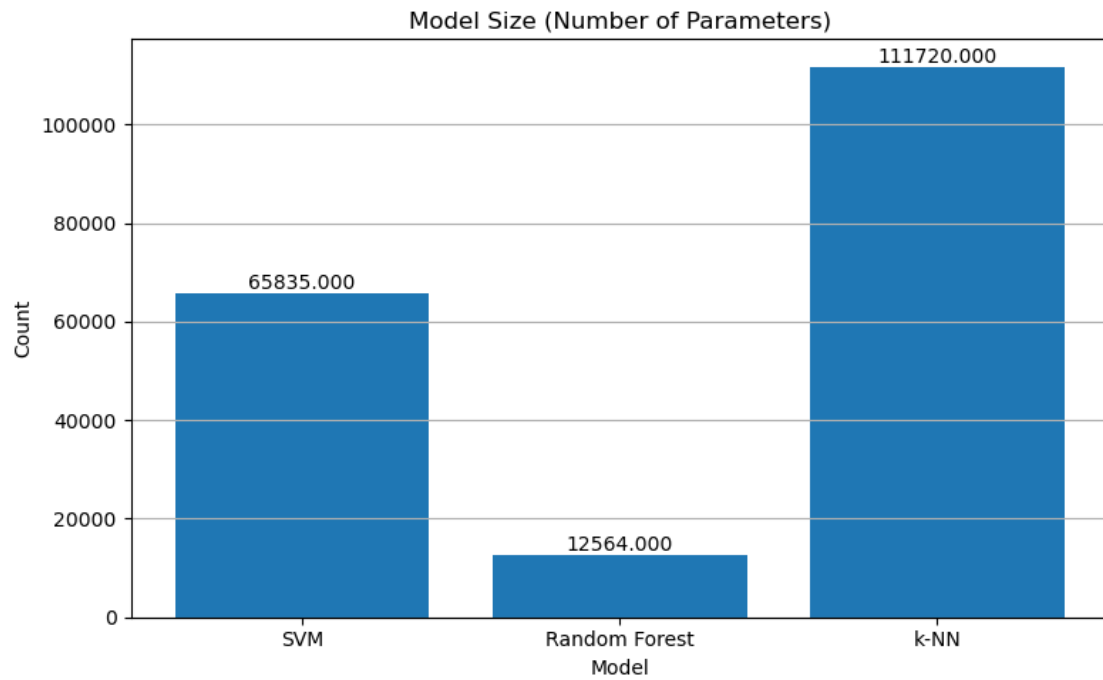
1	0.9950	6.7604	0.0006
2	0.9938	0.0697	0.0001

	Model Parameters	Memory Usage (MB)
0	65835	0.5245
1	12564	1.3284
2	111720	0.4387









1.0.1 Per-Class Metrics — SVM

	precision	recall	f1-score	support
1_Diving	1.0000	0.8333	0.9091	12.0000
2_JumpRope	0.9231	1.0000	0.9600	12.0000
3_SkyDiving	0.9091	0.8333	0.8696	12.0000
4_Surfing	0.8462	0.9167	0.8800	12.0000
5_Swing	0.9231	1.0000	0.9600	12.0000
accuracy	0.9167	0.9167	0.9167	0.9167
macro avg	0.9203	0.9167	0.9157	60.0000
weighted avg	0.9203	0.9167	0.9157	60.0000

1.0.2 Per-Class Metrics — Random Forest

	precision	recall	f1-score	support
1_Diving	0.9091	0.8333	0.8696	12.0000
2_JumpRope	0.9231	1.0000	0.9600	12.0000
3_SkyDiving	1.0000	0.9167	0.9565	12.0000
4_Surfing	0.9231	1.0000	0.9600	12.0000
5_Swing	0.9167	0.9167	0.9167	12.0000
accuracy	0.9333	0.9333	0.9333	0.9333
macro avg	0.9344	0.9333	0.9326	60.0000
weighted avg	0.9344	0.9333	0.9326	60.0000

1.0.3 Per-Class Metrics — k-NN

	precision	recall	f1-score	support
1_Diving	0.9167	0.9167	0.9167	12.0000
2_JumpRope	0.9231	1.0000	0.9600	12.0000
3_SkyDiving	0.9091	0.8333	0.8696	12.0000
4_Surfing	0.9167	0.9167	0.9167	12.0000
5_Swing	1.0000	1.0000	1.0000	12.0000
accuracy	0.9333	0.9333	0.9333	0.9333
macro avg	0.9331	0.9333	0.9326	60.0000
weighted avg	0.9331	0.9333	0.9326	60.0000

1.1 Key Observations

	Observation \
0	Highest Accuracy
1	Best Overall Balance (Macro F1-Score)
2	Precision-Recall Characteristics
3	Model Characteristics
	Details
0	
	Random Forest (0.933)

```

1                                     k-NN (0.933)
2                                     Random
  Forest: Precision=0.934, Recall=0.933
3 SVM: margin-based classifier\nRandom Forest: ensemble-based and robust\nk-NN:
  instance-based and distance-sensitive

```

1.1.1 Final Conclusion

The comparative evaluation of classical machine learning models reveals distinct strengths across different performance criteria.

Based on macro-averaged evaluation: - **Highest Accuracy:** Random Forest - **Best Precision (Macro):** Random Forest - **Best Recall (Macro):** Random Forest - **Best Overall Balance (Macro F1-Score):** Random Forest - **Best Discriminative Capability (Macro ROC-AUC):** Random Forest

Among the evaluated models, **Random Forest** demonstrates the most balanced and consistent performance across classes. These results indicate that no single model dominates all metrics, and model selection should be guided by task-specific requirements.

Comparative CSV saved → /home/jovyan/va-assignment-1/Student_2024ab05275_Video_Classification/results/comparative_analysis.csv

Full Comparative Analysis Completed Successfully.

[]: